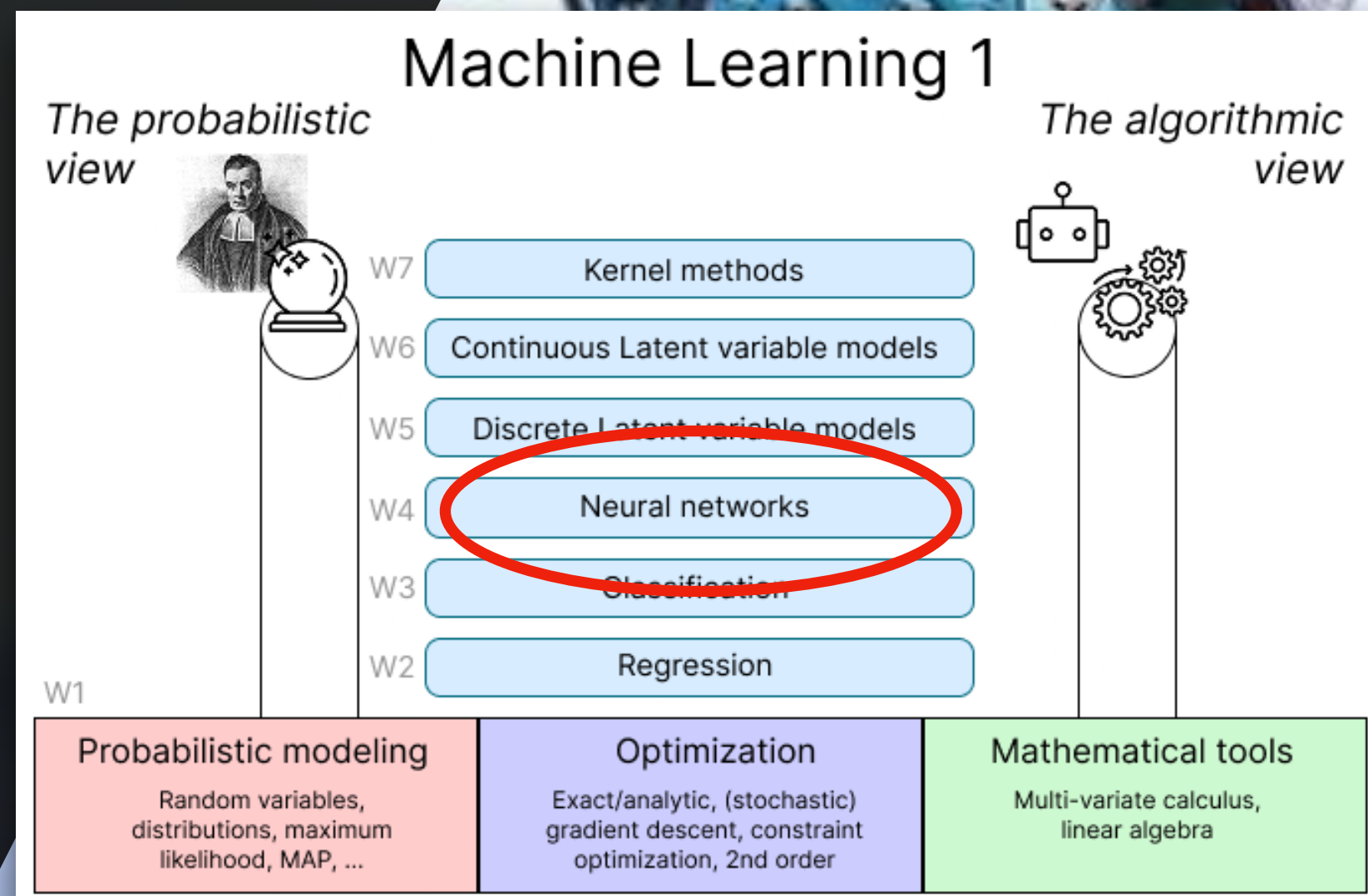




Machine Learning 1

Lecture 8 - Neural Networks

Erik Bekkers



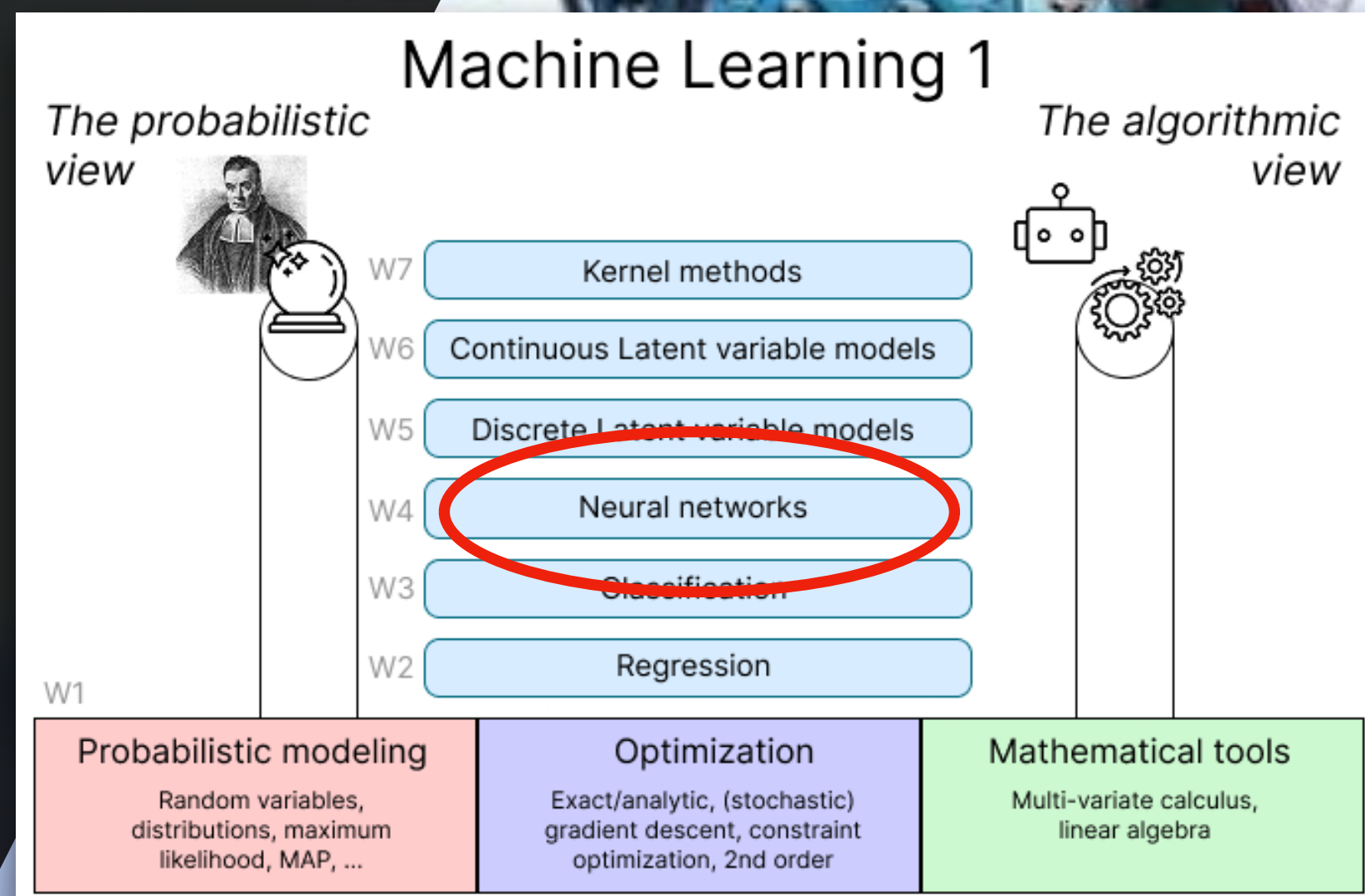


Machine Learning 1

Lecture 8.1 - Supervised Learning Neural Networks

Erik Bekkers

(Bishop 5.1)



Fixed Basis Functions

Dataset: inputs $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_N)^T$ and targets $\mathbf{t} = (t_1, \dots, t_N)^T$

Previously:

learn them
▶ Fixed features: $\phi(\mathbf{x}) = (\phi_0(\mathbf{x}), \dots, \phi_{M-1}(\mathbf{x}))^T$, $\phi_0(\mathbf{x}) = 1$

▶ Linear regression: $y(\mathbf{x}, \mathbf{w}) = \mathbf{w}^T \phi(\mathbf{x})$ $t_n \in \mathbb{R}$

$p(C_k | \mathbf{x}; \mathbf{w}) \in [0, 1]$

▶ Classification: $y(\mathbf{x}, \mathbf{w}) = f(\mathbf{w}^T \phi(\mathbf{x}))$ $t_n \in \{0, 1\}$

$\nwarrow$ sigmoid / soft-max
 f : nonlinear activation function

Neural Networks: Adaptive Basis Functions

Dataset: inputs $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_N)^T$ and targets $\mathbf{t} = (t_1, \dots, t_N)^T$

$$\underline{t} \rightarrow \underline{x}_i \in \mathbb{R}^D$$

$$\underline{W}^{(1)} = \begin{pmatrix} 1 & 1 & 1 \\ \underline{w}_1^{(1)} & \underline{w}_2^{(1)} & \dots & \underline{w}_m^{(1)} \\ 1 & 1 & 1 \end{pmatrix}$$

Neural networks:

- ▶ Create flexible non-linear features and learn them!

$$\phi_m(\mathbf{x}, \mathbf{w}_m^{(1)}) = h((\mathbf{w}_m^{(1)})^T \mathbf{x}) = h\left(\sum_{d=1}^D w_{md}^{(1)} x_d\right)$$

$$\underline{\phi} = \begin{pmatrix} \phi_1 \\ \phi_2 \\ \vdots \end{pmatrix} = h(\underline{W}^{(1)T} \underline{x})$$

- ▶ Regression:

$$y(\mathbf{x}, \mathbf{W}^{(1)}, \mathbf{w}^{(2)}) = \sum_{m=1}^M w_m^{(2)} h\left(\sum_{d=1}^D w_{md}^{(1)} x_d\right) = \underline{w}^{(2)T} \underline{\phi}(\mathbf{x}, \mathbf{w}^{(1)})$$

- ▶ Classification:

$$y(\mathbf{x}, \mathbf{W}^{(1)}, \mathbf{w}^{(2)}) = f(\underline{w}^{(2)T} h(\underline{W}^{(1)T} \underline{x}))$$

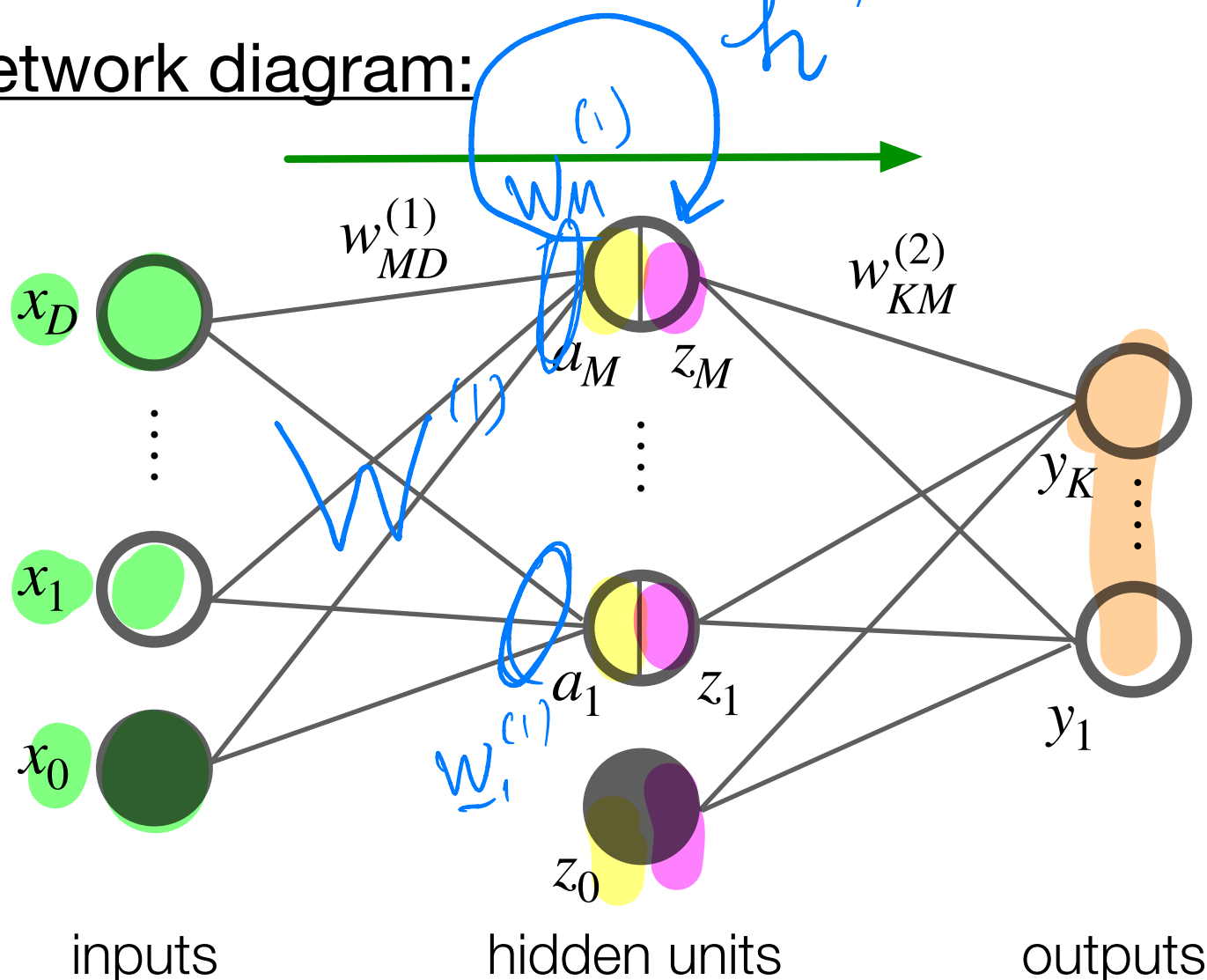
logistic sigmoid / softmax

Multilayer Perceptron (MLP): 2 layers

Model:

$$y_k(\mathbf{x}, \mathbf{W}^{(1)}, \mathbf{W}^{(2)}) = h^{(2)} \left(\sum_{m=0}^M w_{km}^{(2)} h^{(1)} \left(\underbrace{\sum_{d=0}^D w_{md}^{(1)} x_d}_{a_m} \right) \right)$$

Network diagram:



Input units

$$\underline{x} = \begin{pmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \end{pmatrix} \in \mathbb{R}^{D+1}$$

Activations

$$\underline{a} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_M \end{pmatrix} = \mathbf{W}^{(1)\top} \underline{x} \in \mathbb{R}^M$$

Hidden units

$$z_m = h(a_m)$$

Output units

$$\underline{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_K \end{pmatrix} = \mathbf{W}^{(2)\top} \underline{z} \in \mathbb{R}^K$$

Activation functions

$$h^{(1)}, h^{(2)}$$

Activation functions

$$h(a) = \sin(wa)$$

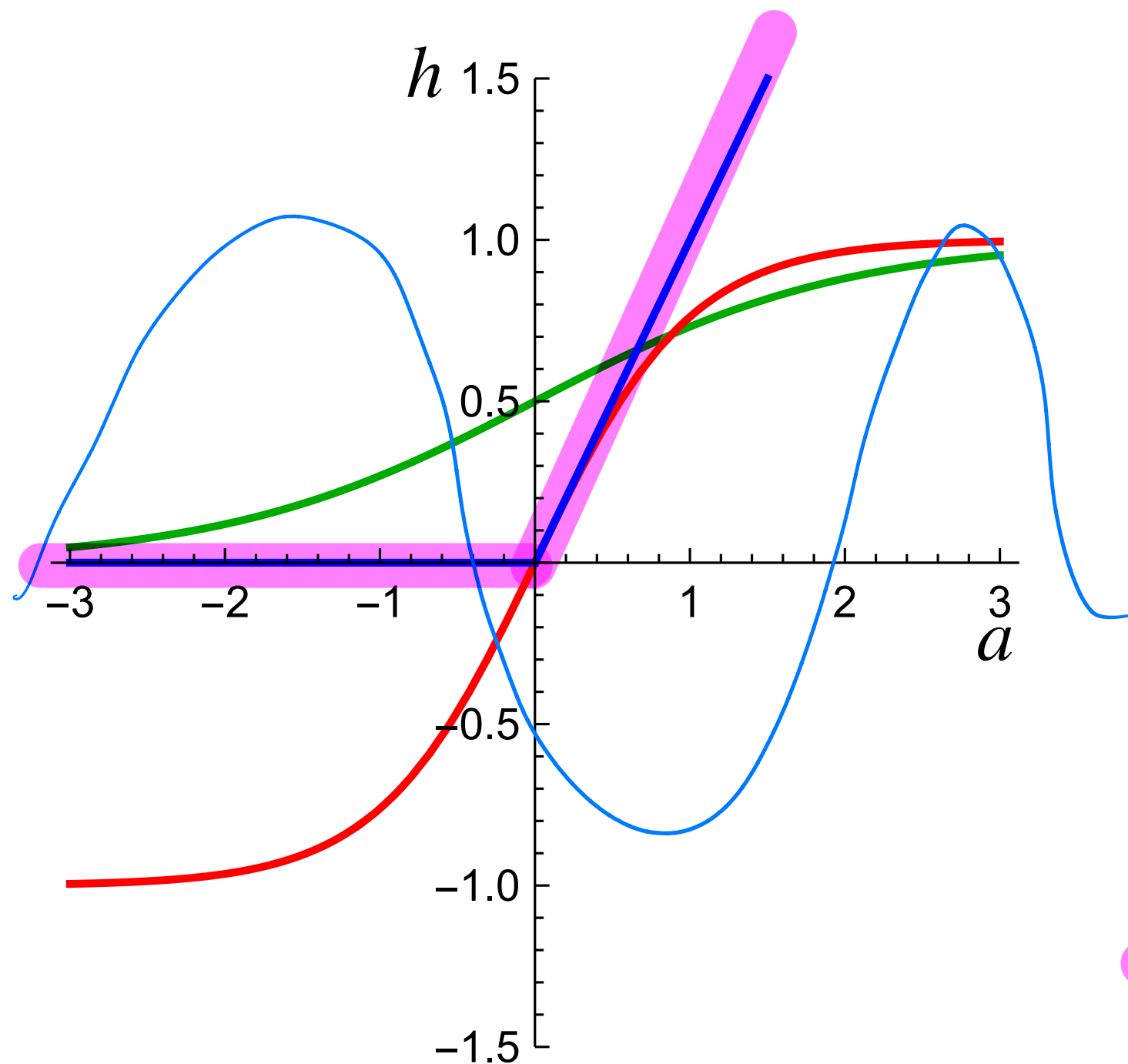


Figure: Popular activation functions.

Green: *Logistic sigmoid*

$$h(a) = \sigma(a) = \frac{1}{1 + e^{-a}}$$

Red: *hyperbolic tangent*

$$h(a) = \tanh(a)$$

Blue: *Rectified linear unit*

$$h(a) = \text{ReLU}(a) = \max(0, a)$$

(Rectified Linear Unit)

Why activation funct,
are needed?

$$h\left(\underbrace{W^{(3)T} h\left(W^{(2)T} h\left(W^{(1)T} \underline{x}\right)\right)}_{M'' \leftarrow M' \leftarrow M \leftarrow D}\right)$$

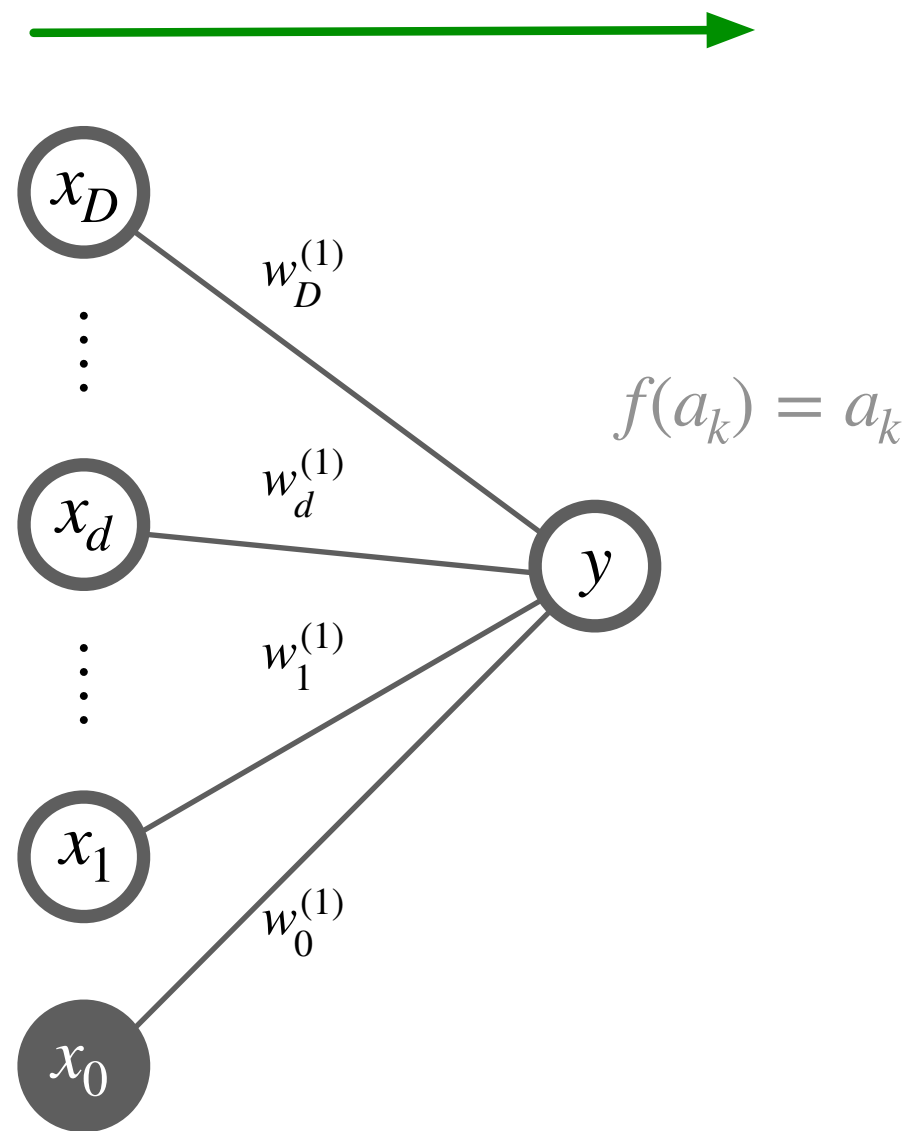
$$M'' \leftarrow M' \leftarrow M \leftarrow D$$

still
linear

$$\underbrace{W^{(3,2,1)T}}_{M'' \times D} \underline{x}$$

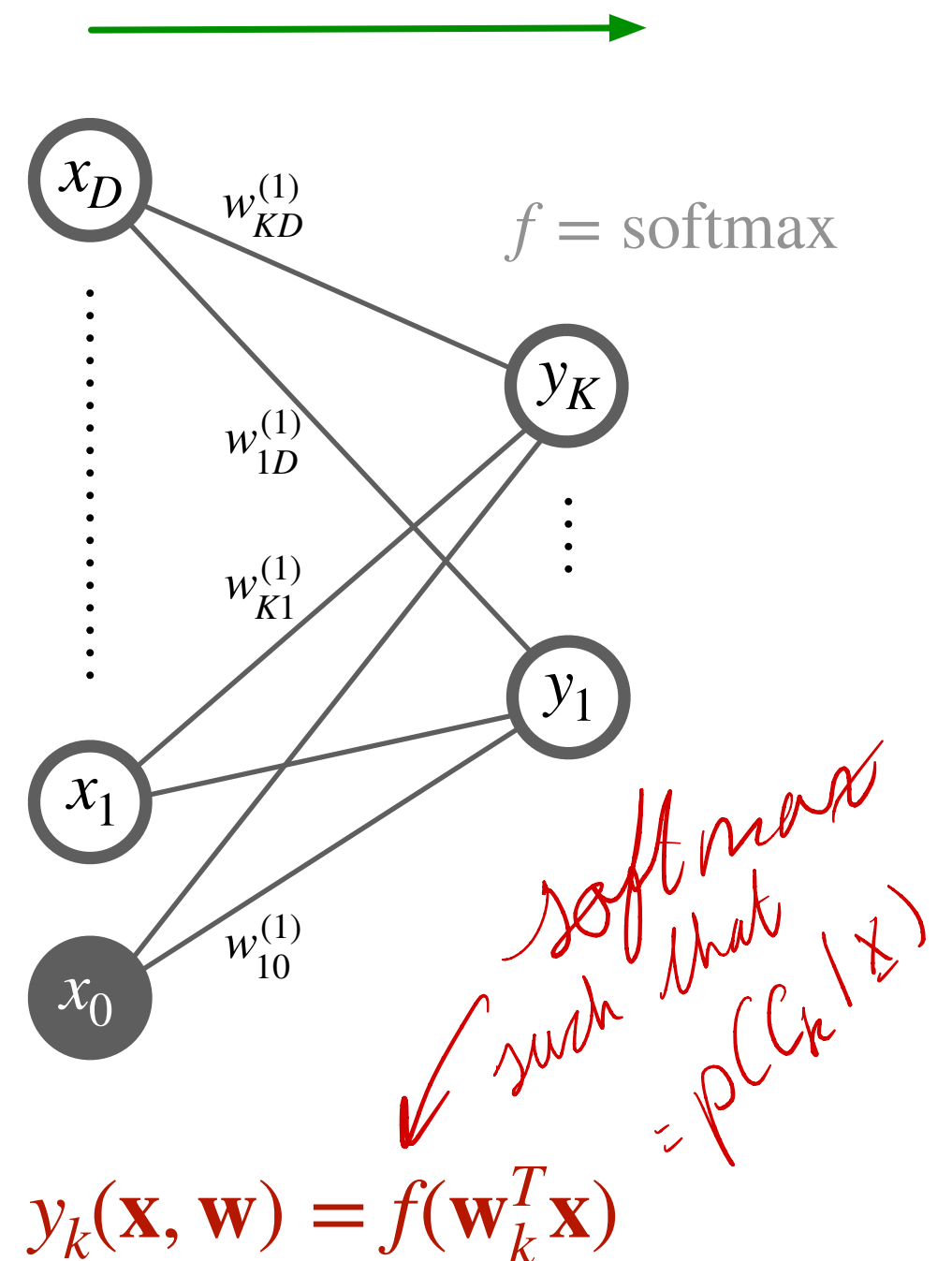
→ without act-fn it is
still effectively one matrix
vector mult.

Linear regression & classification as NN



$$y(\mathbf{x}, \mathbf{w}) = \mathbf{w}^T \mathbf{x}$$

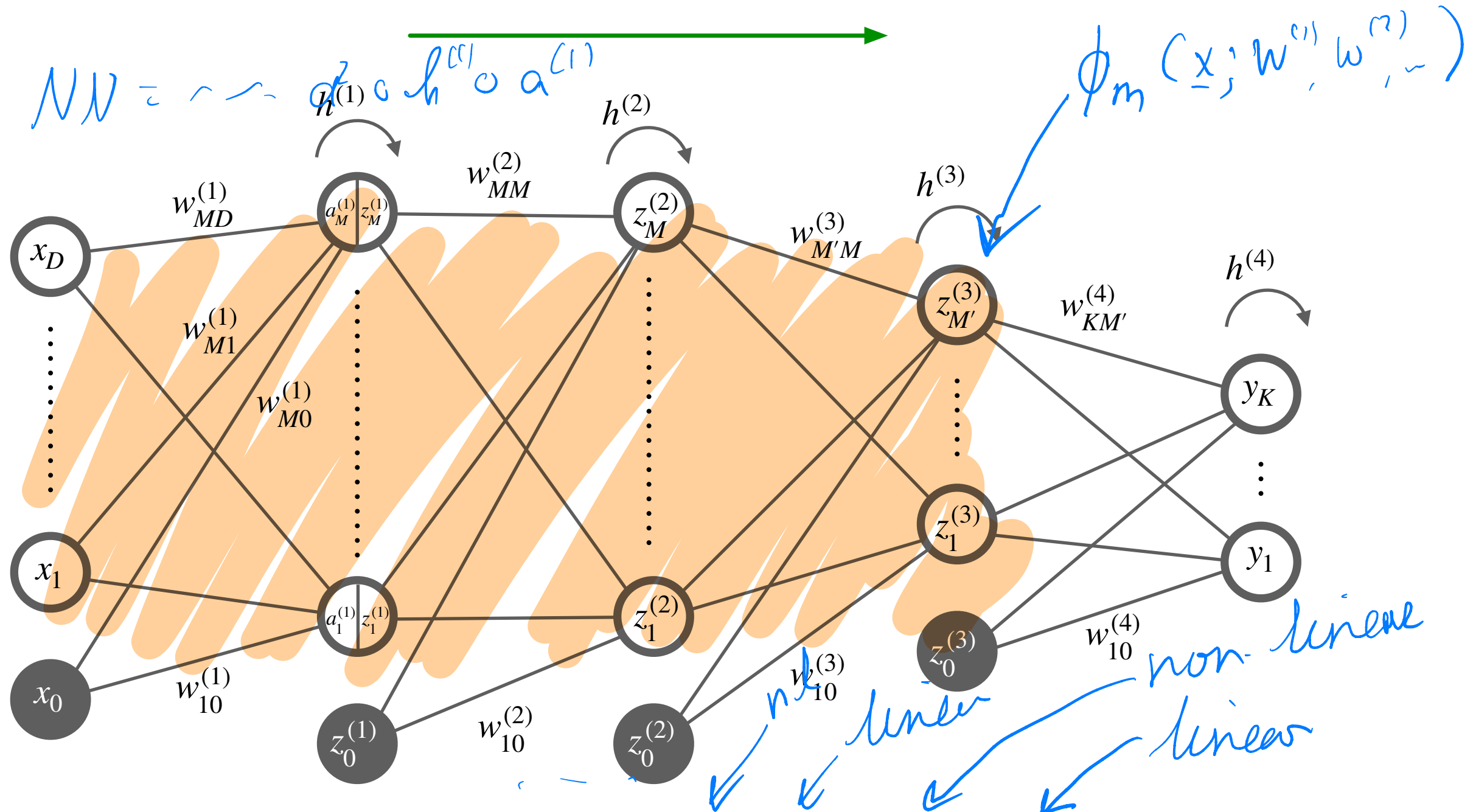
Figure: Linear regression as 1-layer NN



$$y_k(\mathbf{x}, \mathbf{w}) = f(\mathbf{w}_k^T \mathbf{x})$$

Figure: Linear Classification with K classes as 1-layer NN

Feed-Forward Networks: Multiple layers



$$y_k(\mathbf{x}, \mathbf{w}) = h^{(4)}(\mathbf{a}^{(4)}(h^{(3)}(\mathbf{a}^{(3)}(h^{(2)}(\mathbf{a}^{(2)}(h^{(1)}(\mathbf{a}^{(1)}(\mathbf{x})))))))$$

$$= h^{(4)} \circ \mathbf{a}^{(4)} \circ h^{(3)} \circ \mathbf{a}^{(3)} \circ h^{(2)} \circ \mathbf{a}^{(2)} \circ h^{(1)} \circ \mathbf{a}^{(1)}(\mathbf{x})$$

Figure: 4 layer network. Number of layers = number of layers of adaptive weights.

Feed-Forward Networks: Skip Connections

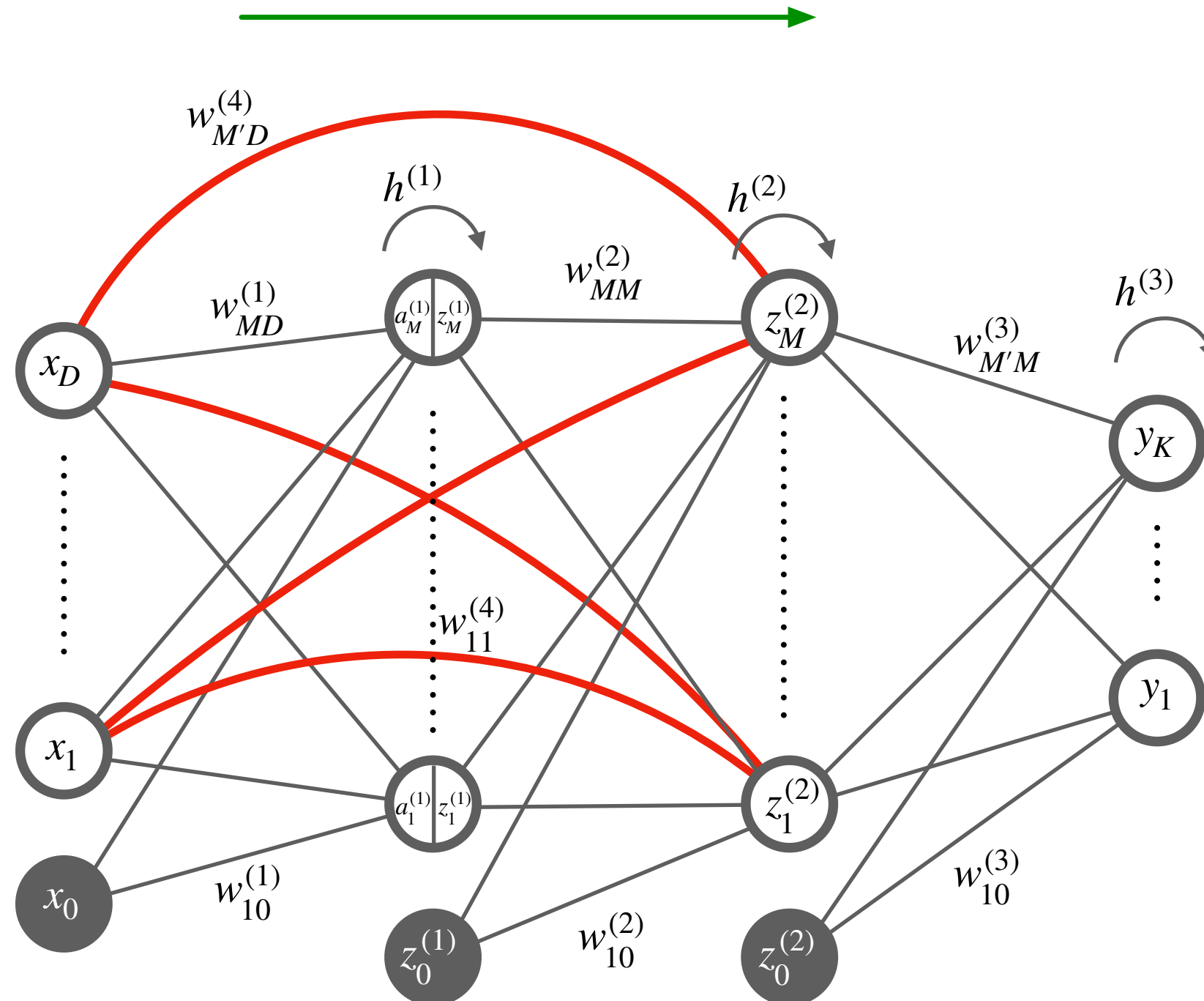


Figure: 3 layer feed-forward net with skip connections

Feed-Forward Networks: Sparse Connections

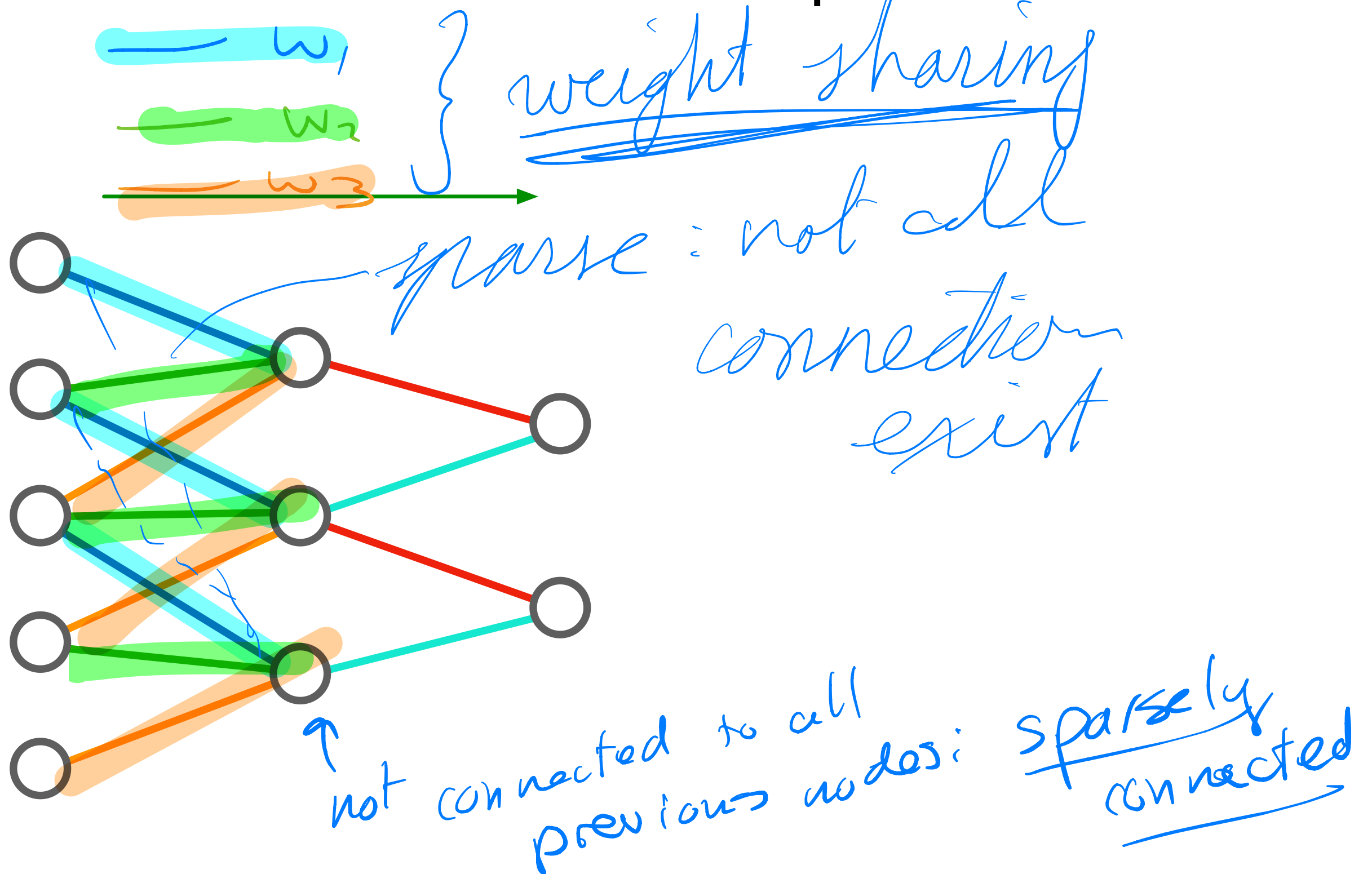
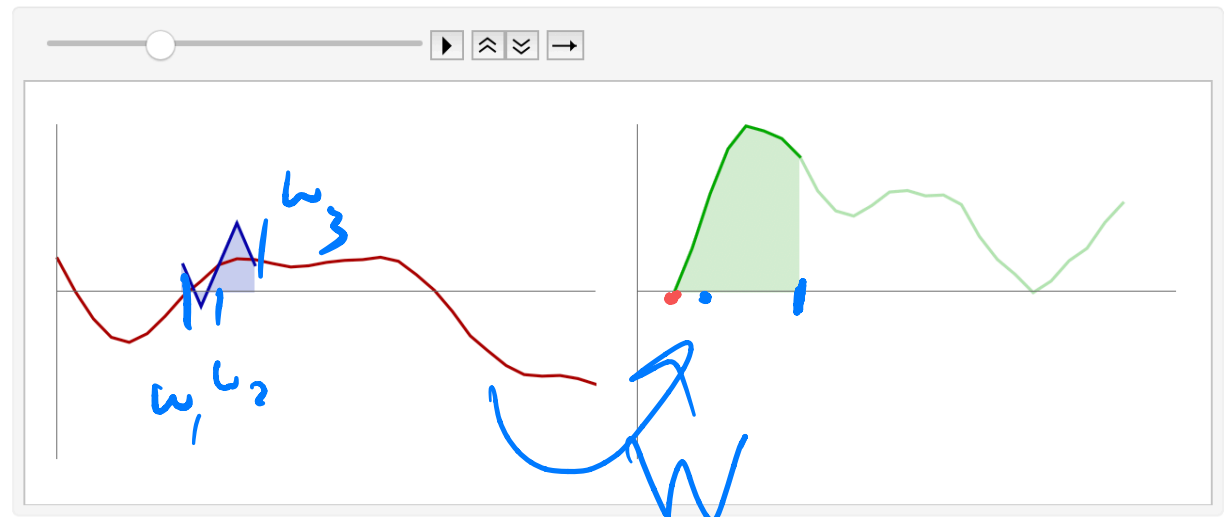
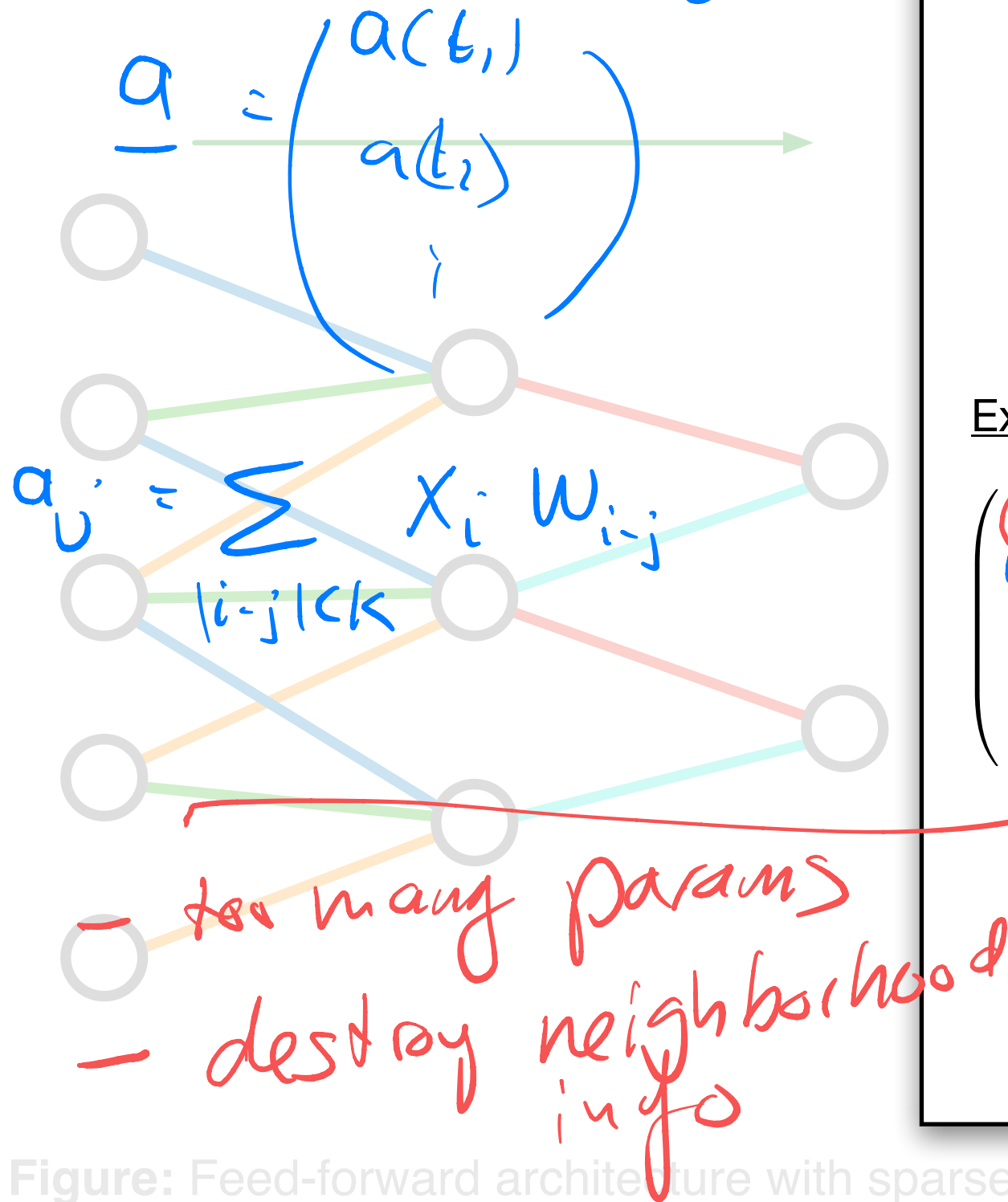


Figure: Feed-forward architecture with sparse connections. With "special weight sharing --> Convolutional Neural Nets (Le Cun et al 1989)" layer is equivariant

Feed-Forward Networks: Sparse Connections

Here are a, x treated as signals



$$\star w = a$$

Example: 1D convolution = sparse + weightsharing:

$$\begin{pmatrix} a_1 \\ a_2 \\ a_2 \\ \vdots \\ a_M \end{pmatrix} = \begin{pmatrix} w_{11} & w_{12} & w_{13} & w_{14} & w_{15} & \dots \\ w_{21} & w_{22} & w_{23} & w_{24} & w_{25} & \dots \\ w_{31} & w_{32} & w_{33} & w_{34} & w_{35} & \dots \\ w_{41} & w_{42} & w_{43} & w_{44} & w_{45} & \dots \\ \vdots & \vdots & \vdots & \ddots & \ddots & \ddots \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_D \end{pmatrix}$$

$$= \begin{pmatrix} w_1 & w_2 & w_3 & 0 & 0 & \dots \\ 0 & w_1 & w_2 & w_3 & 0 & \dots \\ 0 & 0 & w_1 & w_2 & w_3 & \dots \\ 0 & 0 & 0 & w_1 & w_2 & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_D \end{pmatrix}$$

Figure: Feed-forward architecture with sparse connections. With special weight sharing --> Convolutional Neural Nets (Le Cun et al 1989)

General Feed-Forward Architectures

- ◆ Each unit (hidden & output) in feed-forward architectures computes a function of the form

$$z_m = h \left(\sum_j w_{mj} z_j \right)$$

- ◆ No closed directed cycles!

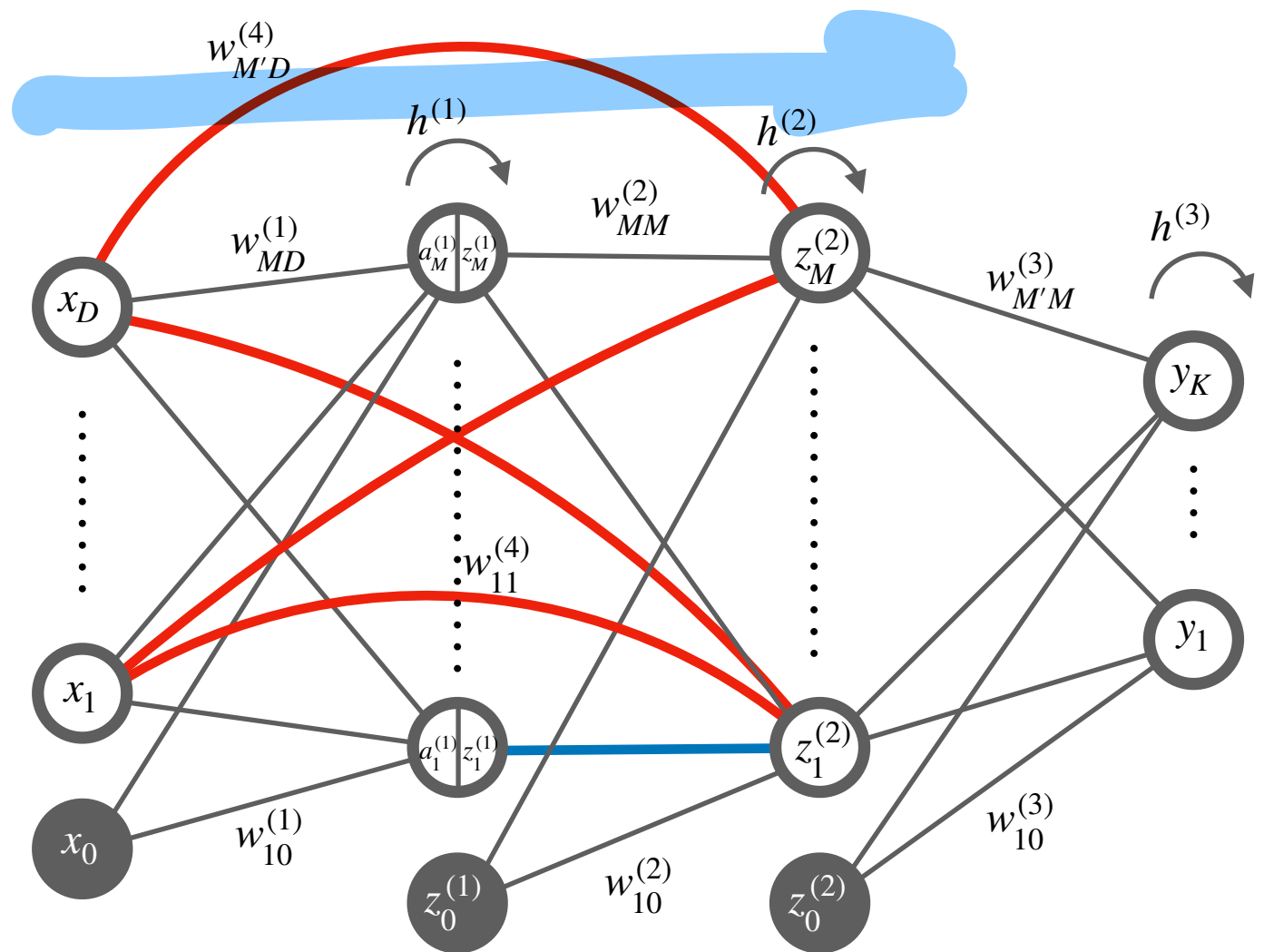


Figure: example of general feed-forward architecture

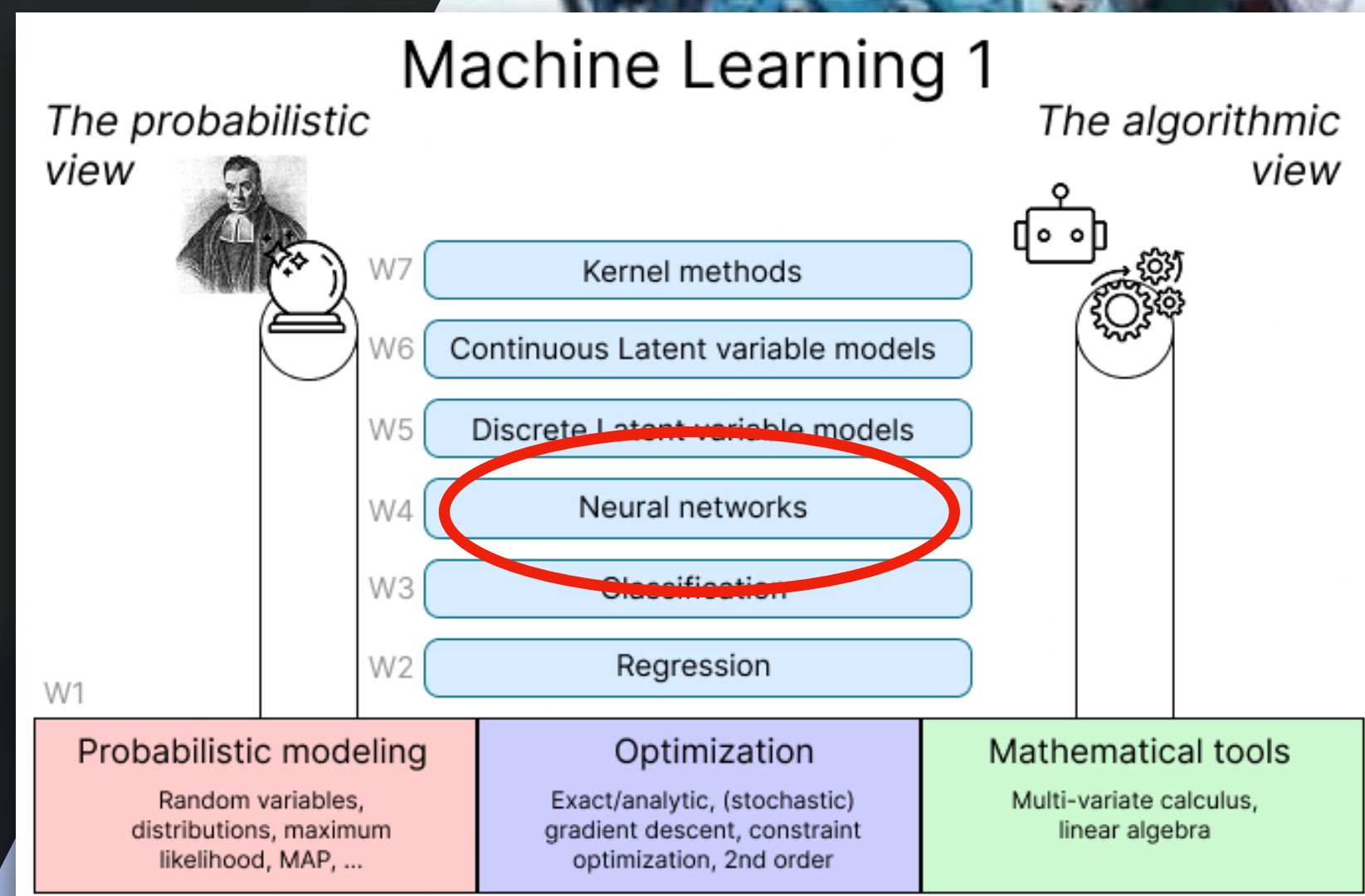


Machine Learning 1

Lecture 8.2 - Supervised Learning
Neural Networks - Universal Approximators

Erik Bekkers

(Bishop 5.1)



NN: Universal Approximators

Theorem: Universal Approximators

- Let f be any continuous function on a compact area of $\mathbb{R}^D$
- Let h any fixed analytic function *which is not polynomial* (e.g. logistic function, tanh function, ...).

$$h(x) = x^2$$

Given any small number $\epsilon > 0$ of an acceptable error, we can find a number M and weights $\mathbf{w}^{(2)} \in \mathbb{R}^M$ and $\mathbf{W}^{(1)} \in \mathbb{R}^{M \times D}$ such that:

↑
width

$$|f(\mathbf{x}) - y(\mathbf{x}, \mathbf{W}^{(1)}, \mathbf{w}^{(2)})| < \epsilon$$

NN

with y a 2-layer NN:

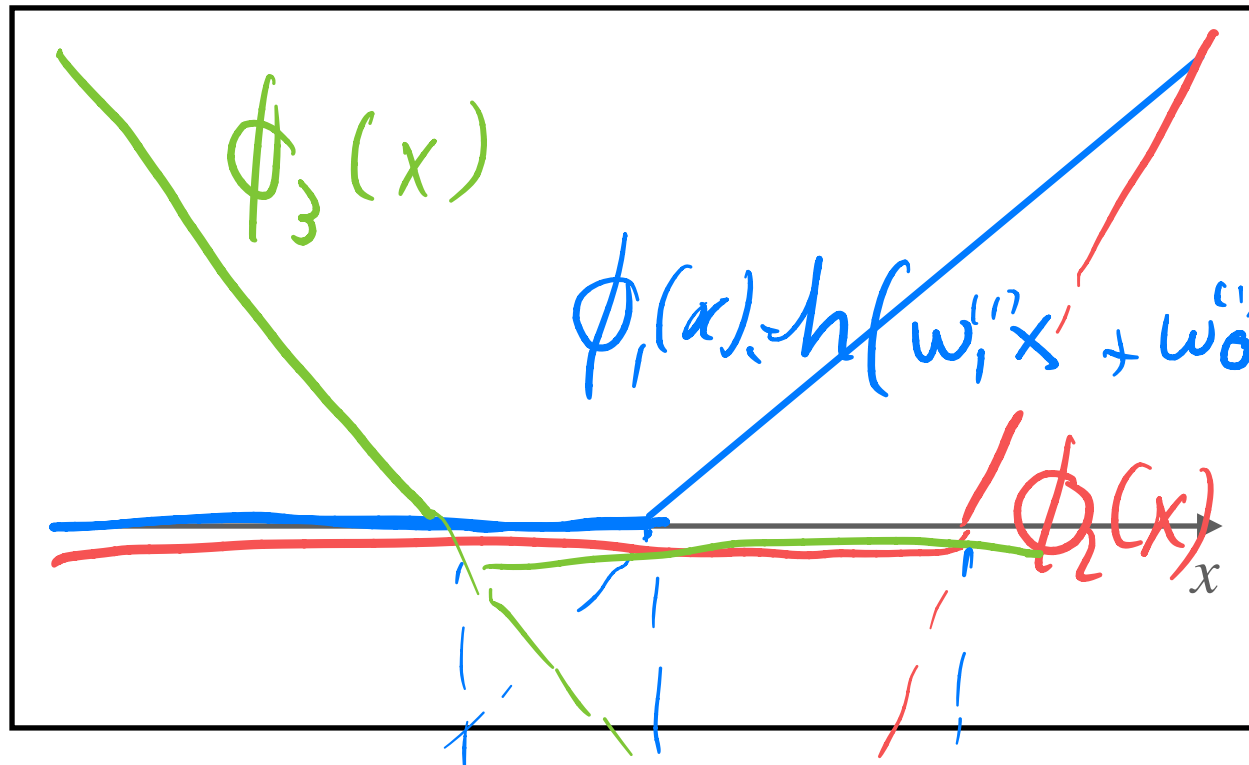
$$y(\mathbf{x}, \mathbf{W}^{(1)}, \mathbf{w}^{(2)}) = \sum_{m=0}^M w_m^{(2)} h \left(\sum_{d=0}^D w_{md}^{(1)} x_d \right)$$

M -dimensional
learned
basis
vector

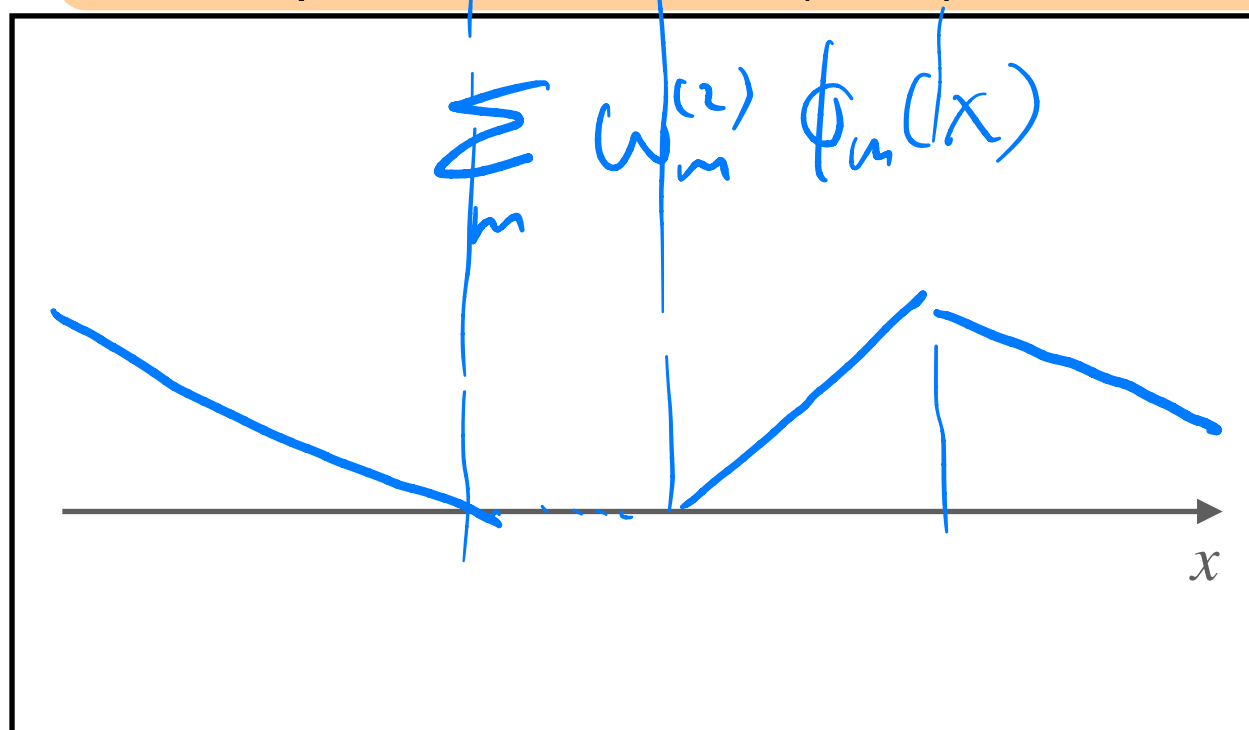
Caution: for smaller ϵ we usually need larger M

Neural Networks with ReLU

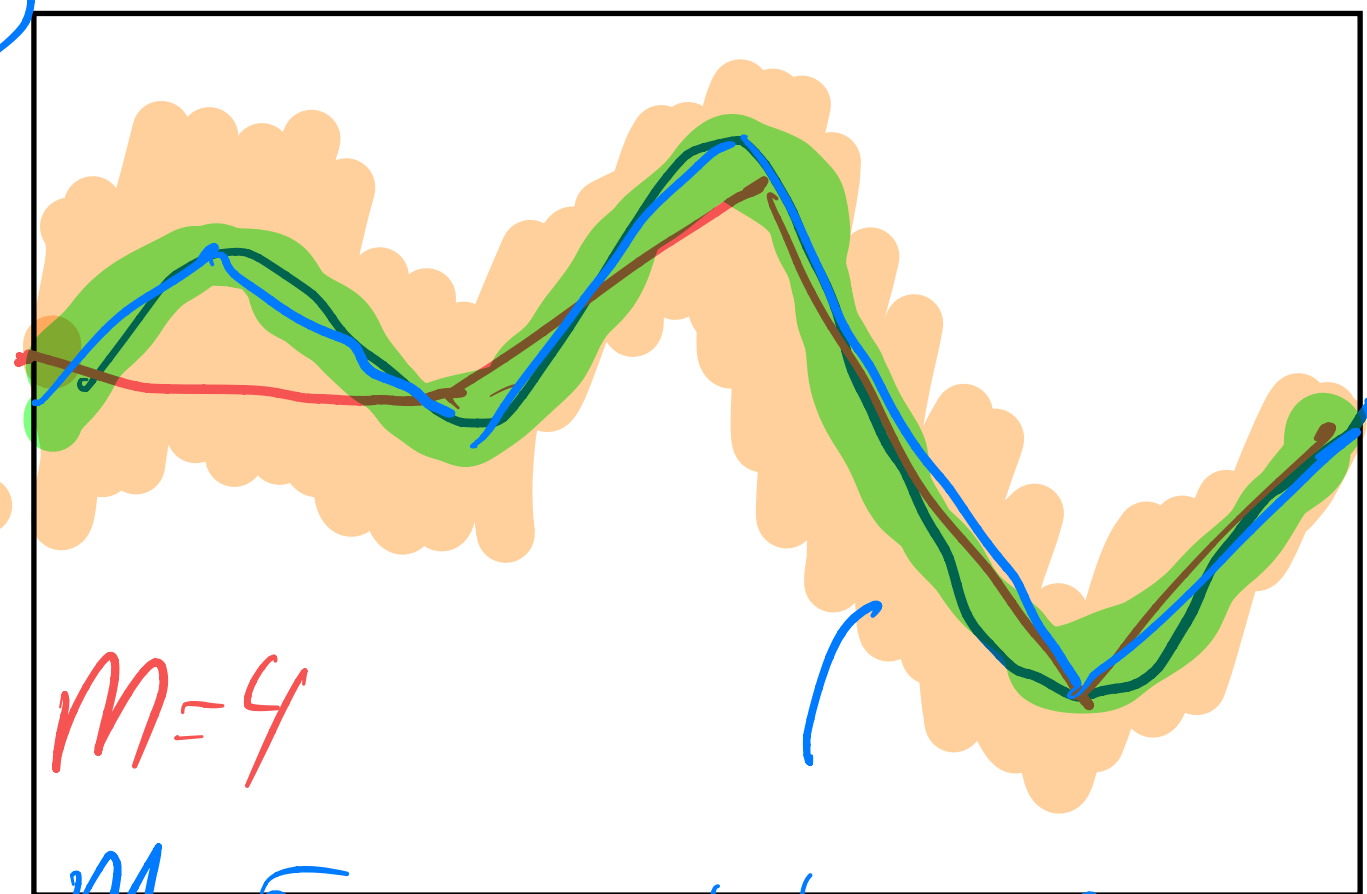
Learned basis functions with ReLU



Leads to piece-wise linear (PWL) functions



Approximation with ReLU NNs/PWL functions



tolerance
 ϵ

Deep Neural Nets and Shallow Neural Nets

- ▶ Take a neural net with L layers.
- ▶ Take a more shallow neural net with $L' < L$ layers.
- ▶ Approximate the deep neural net with shallow neural net up to error ϵ
- ▶ Usually number of required units $M(\epsilon)$ of the shallow net scales exponentially for decreasing ϵ !

Expressive power ReLU networks

- ▶ Expressive power of ReLU-DNN = number of linear regions

$$\# \text{ regions} \approx \text{width}^{\text{depth} \cdot D}$$

- ▶ Polynomial in width, but exponential with depth
- ▶ With fixed network capacity

$$\# \text{ parameters} \approx \text{width}^2 \cdot \text{depth}$$

- ▶ Most expressive power is gained by going deeper with less neurons per layer than staying shallow with more neurons per layer.

Example: Function Approximations

(a) $f(x) = x^2$

(b) $f(x) = \sin(x)$

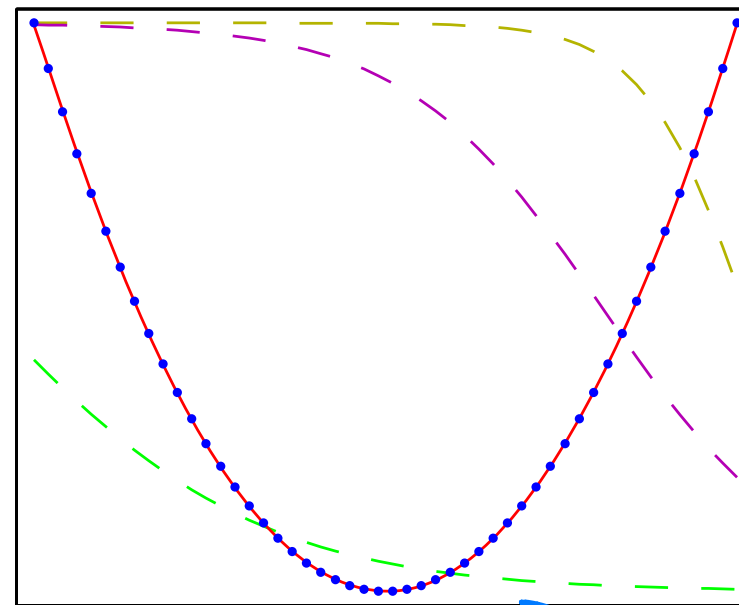
(c) $f(x) = |x|$

(d) $f(x) = H(x)$

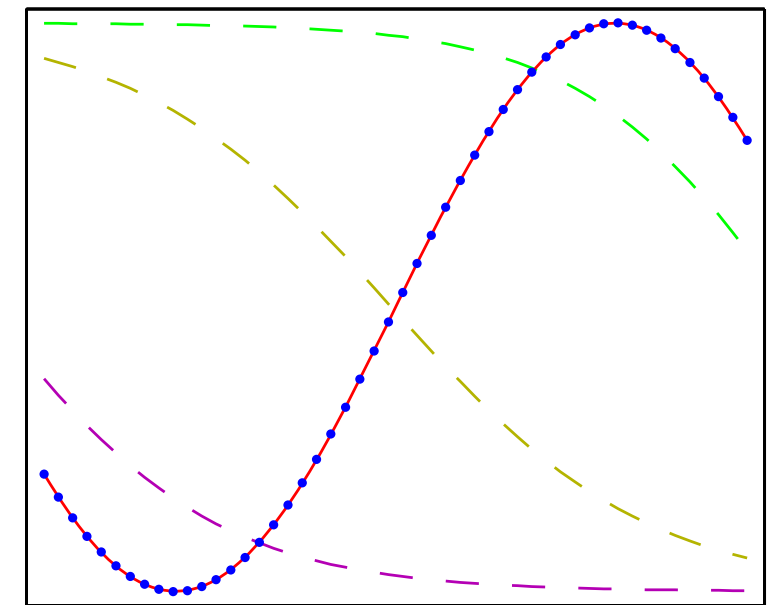
◆ $N=50$ datapoints

◆ MLP: 2 layers, 3 hidden units with tanh activation function. 1 linear output unit.

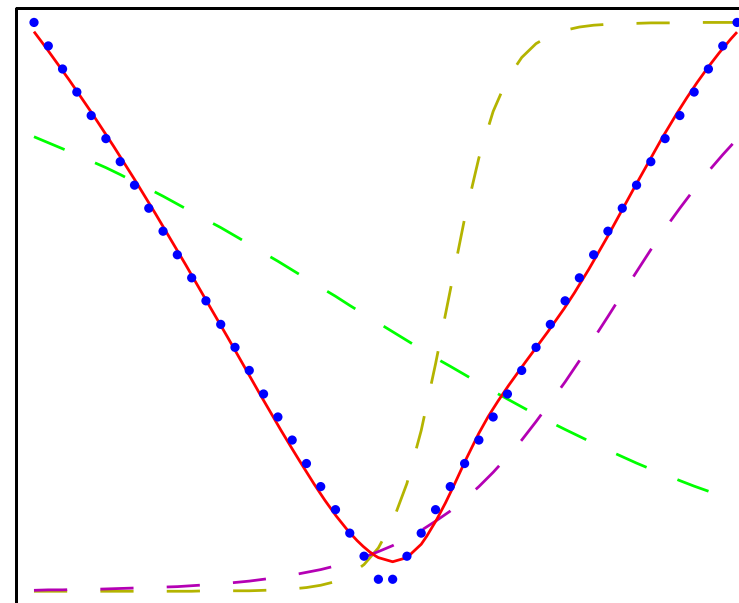
◆ Hidden unit outputs: dashed curves



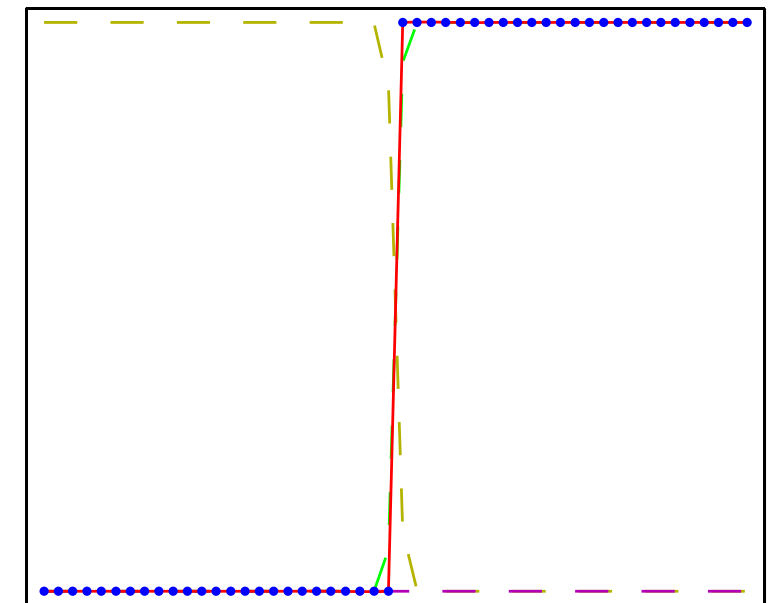
(a)



(b)



(c)

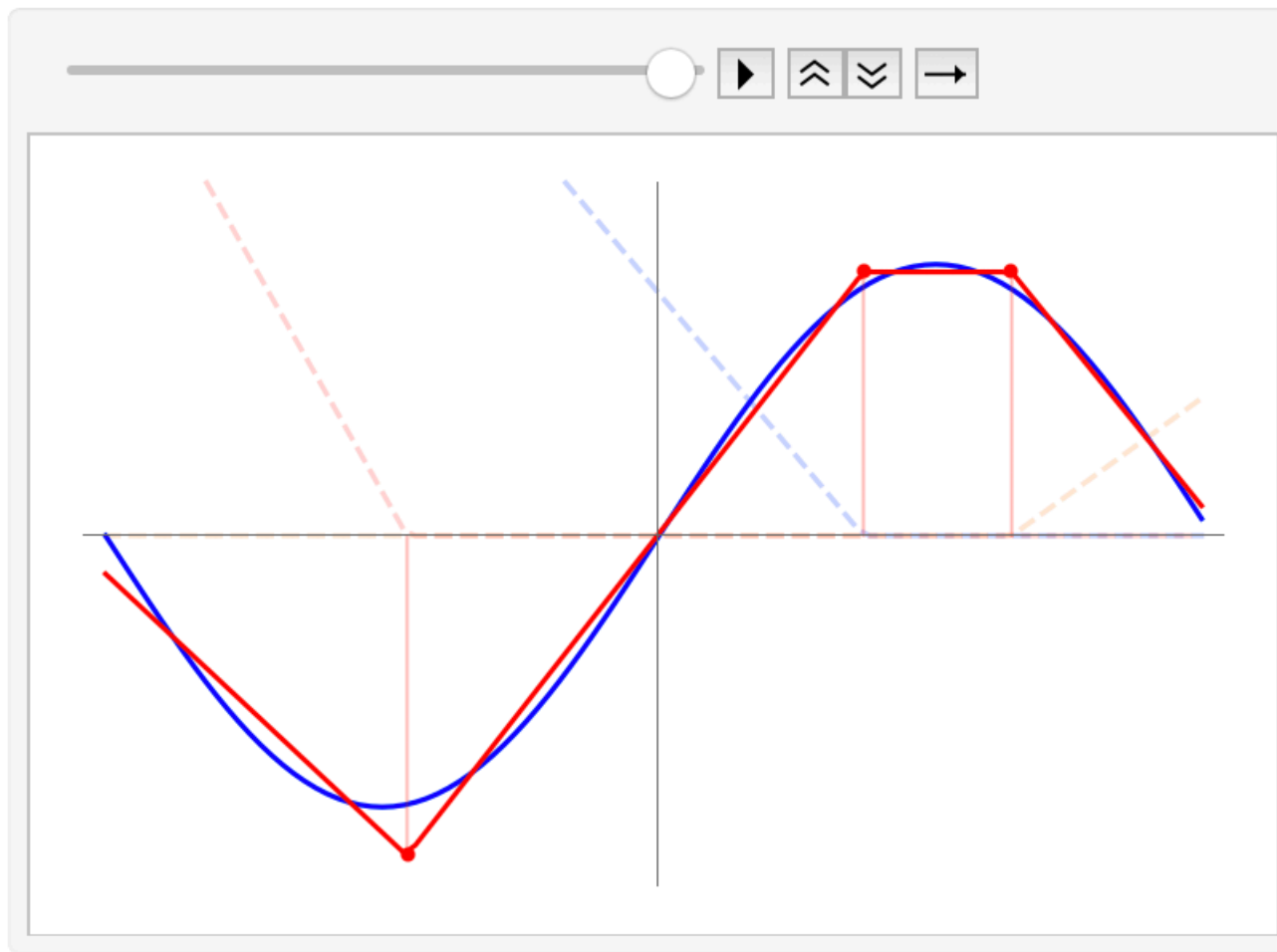


(d)

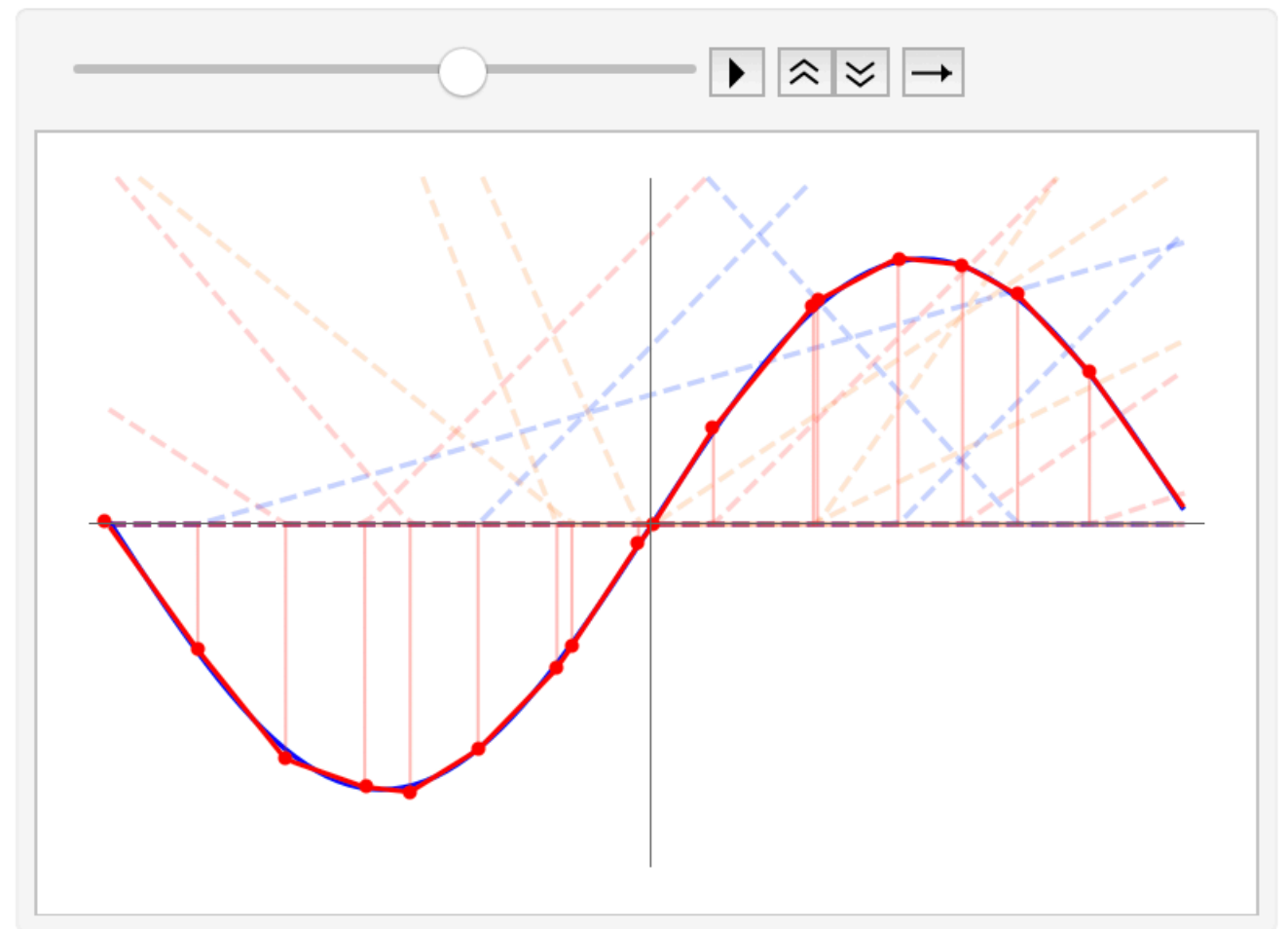
Figure: MLP approximating four different functions (red curves) (Bishop 5.3)

Example: Function Approximations

Piece-wise linear approximation with 2-layer NN



M=3 hidden units



M=20 hidden units

Example: Classification with Neural Nets

MLP:

- ▶ 2 layers
- ▶ # of inputs: 2
- ▶ 2 hidden units with tanh activation function
- ▶ # of outputs: 1 or 2 x_2
- ▶ Output activation function: softmax
- ▶ Red line: MLP decision boundary
- ▶ Green line: optimal decision boundary from synthetic data distribution

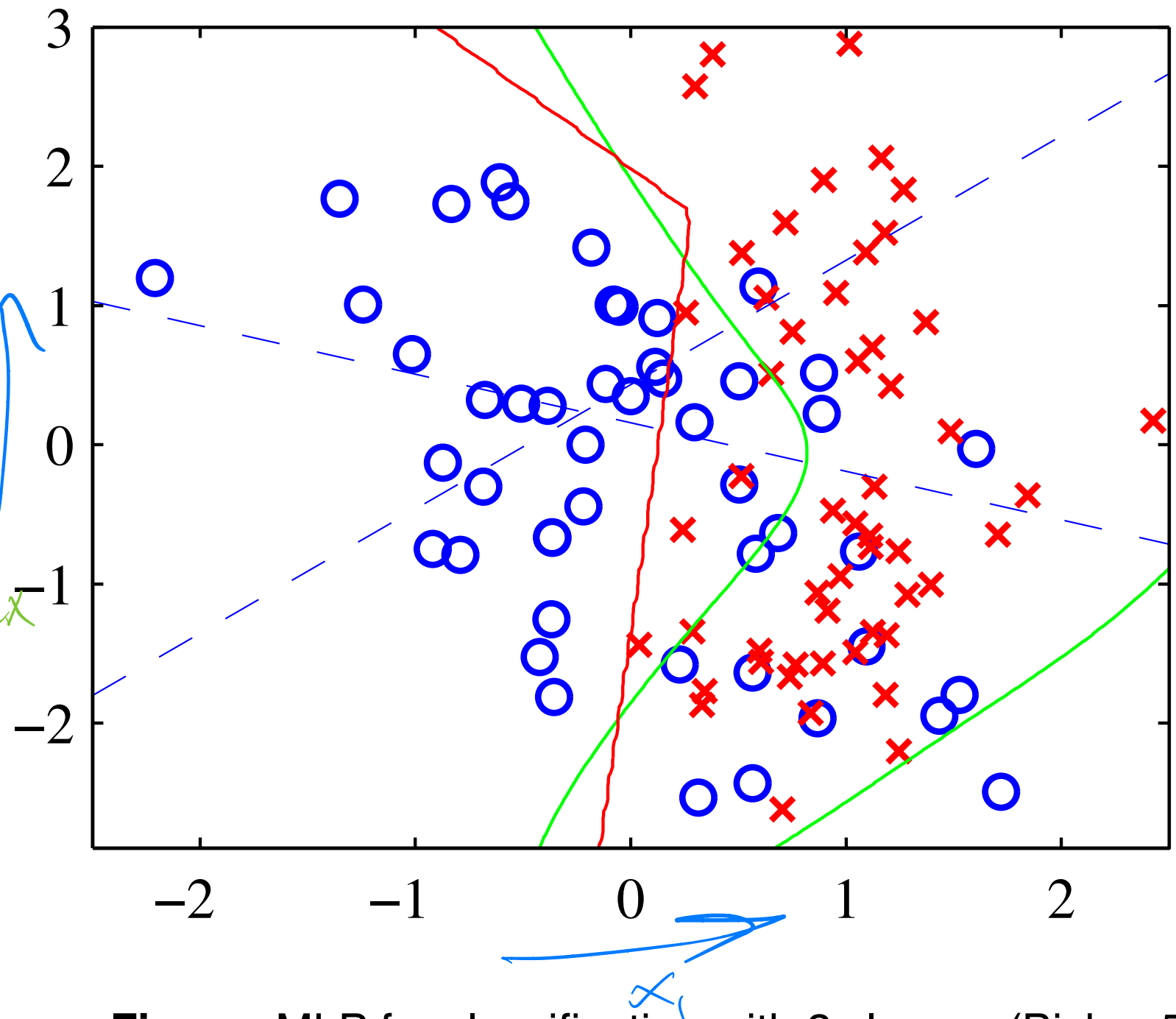


Figure: MLP for classification with 2 classes (Bishop5.4)

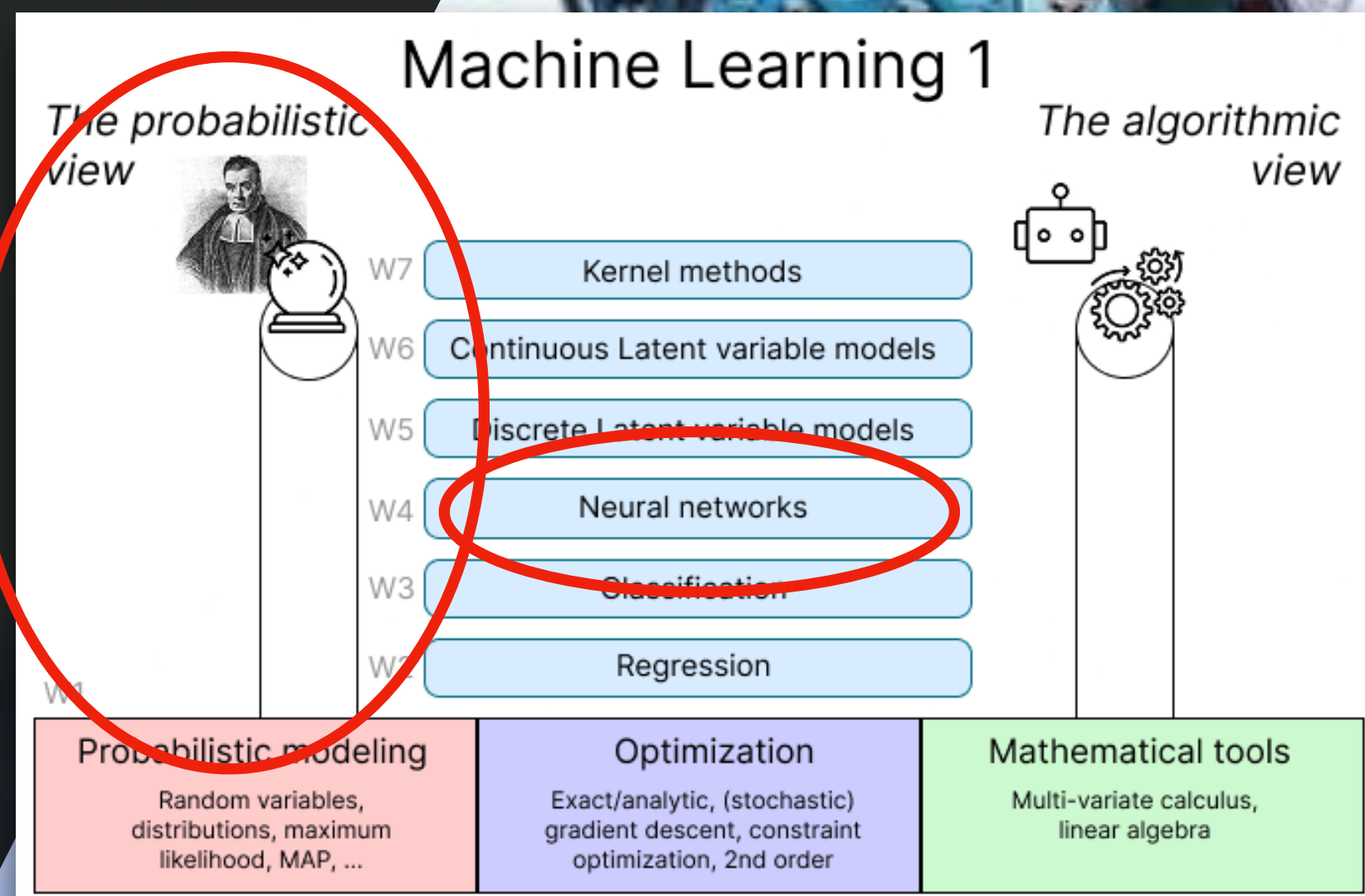


Machine Learning 1

Lecture 8.3 - Supervised Learning
Neural Networks - Losses

Erik Bekkers

(Bishop 5.2.0)



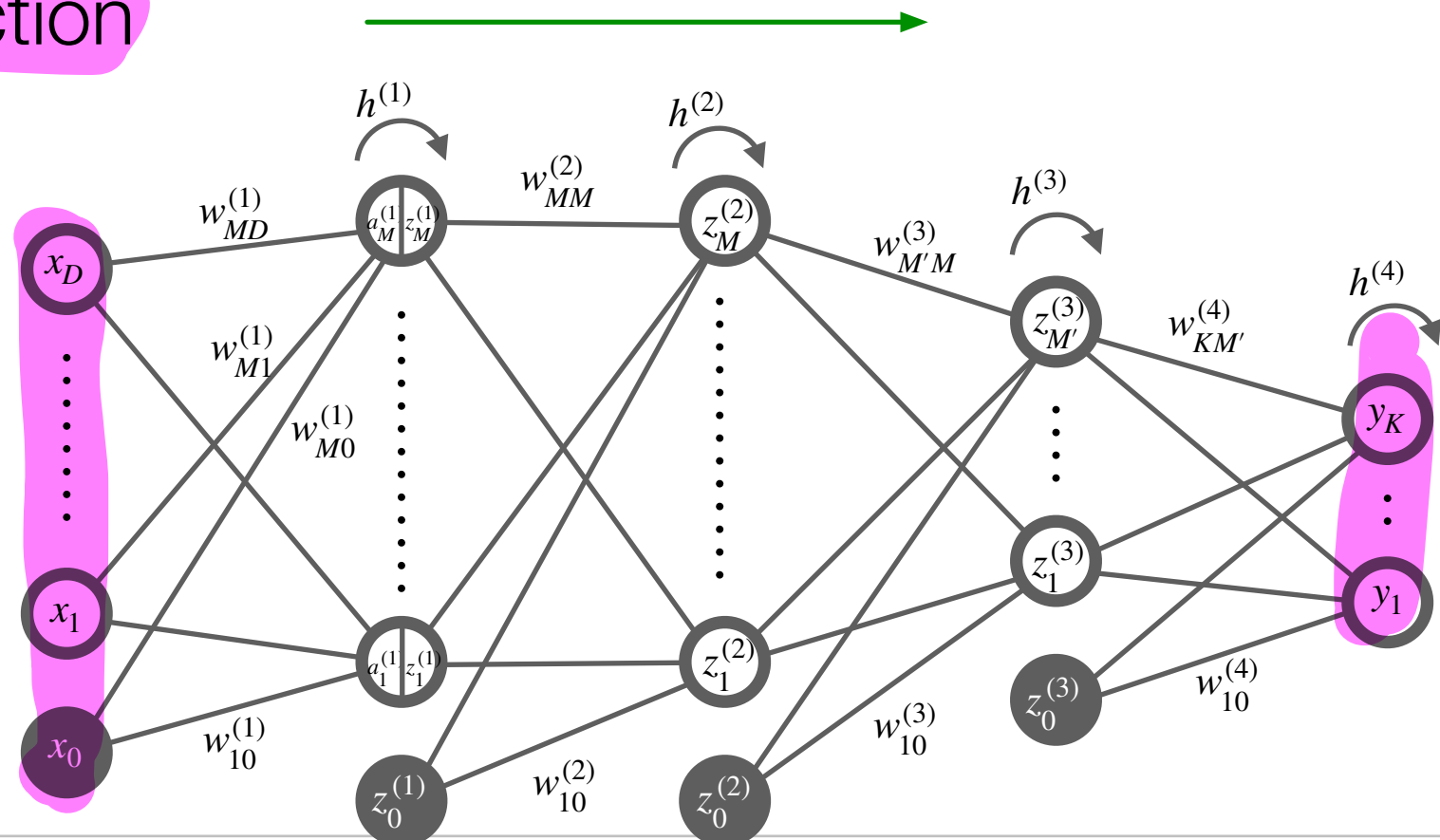
Network Training

- Dataset: inputs $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_N)^T$, $\mathbf{x}_n \in \mathbb{R}^D$
- Use a probabilistic interpretation of the network outputs to choose

1. Number of outputs

2. Output activation function

3. Loss function!



Network Training: Regression

- ▶ Data: inputs $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_N)^T$, and targets $\mathbf{t} = (t_1, \dots, t_N)^T$

- ▶ Assume target distribution:

✓ $p(t | \mathbf{x}, \mathbf{w}) = N(t | y(\mathbf{x}; \mathbf{w}), \beta^{-1})$

- ▶ Single target $\rightarrow$ Single output unit:

✓ $y(\mathbf{x}, \mathbf{w}) = h^{(L)}(a^{out})$

operator as act fun

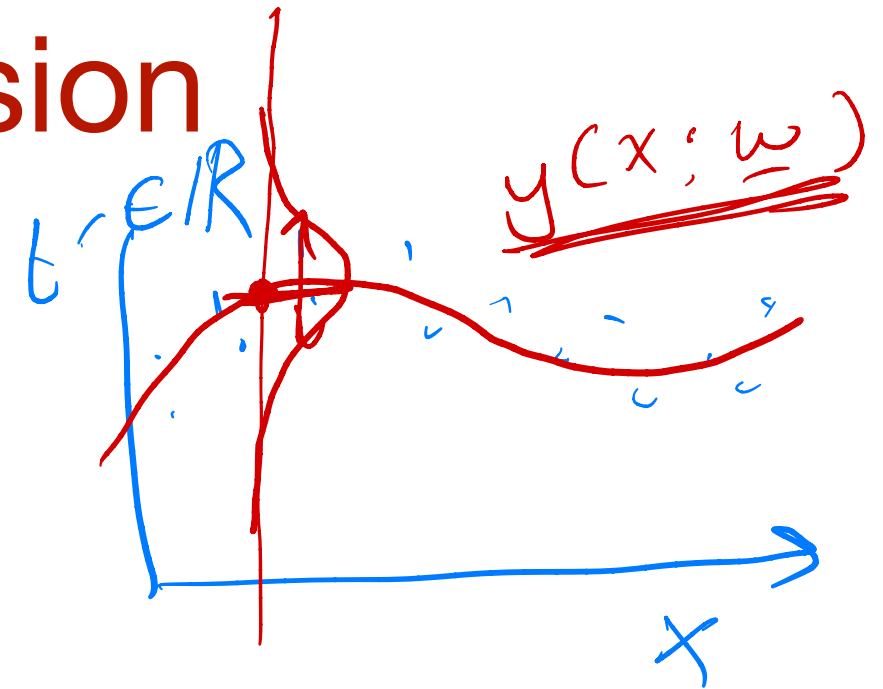
- ▶ Targets are real valued: identity output activation function:

✓ $y(\mathbf{x}, \mathbf{w}) = h^{(L)}(a^{out}) = a^{out}$

- ▶ Maximum Likelihood/minimum negative log likelihood:

$$E(\mathbf{w}) = -\ln p(\mathbf{t} | \mathbf{X}, \mathbf{w}) = \frac{\beta}{2} \sum_{n=1}^N \{y(\mathbf{x}_n, \mathbf{w}) - t_n\}^2 - \frac{N}{2} \ln \beta + \frac{N}{2} \ln 2\pi$$

- ▶ Equivalently: $E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(\mathbf{x}_n, \mathbf{w}) - t_n\}^2$



Network Training: Binary Classification

- ▶ Data: inputs $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_N)^T$, and targets $\mathbf{t} = (t_1, \dots, t_N)^T$

$t=0 \rightarrow C_2$

- ▶ Let $p(t = 1 | \mathbf{x}) = y(\mathbf{x}, \mathbf{w})$ and assume target distribution

Bernoulli

$$p(t | \mathbf{x}, \mathbf{w}) = y(\mathbf{x}, \mathbf{w})^t (1 - y(\mathbf{x}, \mathbf{w}))^{1-t}$$

- ▶ Single target $\rightarrow$ Single output unit:

$$y(\mathbf{x}, \mathbf{w}) = h^{(L)}(a^{out})$$

- ▶ Targets are binary: sigmoid output activation function:

$$y(\mathbf{x}, \mathbf{w}) = h^{(L)}(a^{out}) = \sigma(a^{out}) \in [0, 1]$$

binary cross-entropy loss

- ▶ Maximum Likelihood/minimum negative log likelihood:

$$E(\mathbf{w}) = - \sum_{n=1}^N t_n \ln y(\mathbf{x}_n, \mathbf{w}) + (1 - t_n) \ln(1 - y(\mathbf{x}_n, \mathbf{w}))$$

Network Training: Classification with K classes

- ▶ Data: inputs $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_N)^T$, and targets $\mathbf{T} = (\mathbf{t}_1, \dots, \mathbf{t}_N)^T$

$$\mathbf{t}_n = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}$$

- ▶ Let $p(C_k | \mathbf{x}) = y_k(\mathbf{x}, \mathbf{w})$ and assume target distribution

$$p(\mathbf{t}_n | \mathbf{x}_n, \mathbf{w}) = \prod_{k=1}^K y_k(\mathbf{x}, \mathbf{w})^{t_{nk}}$$

- ▶ K targets $\rightarrow$ K output units:

$$y_k(\mathbf{x}, \mathbf{w}) = h^L(a_k^{\text{out}}) \quad \text{such that}$$

$$\sum_{k=1}^K p(C_k | \mathbf{x}) = 1$$

$$\Leftrightarrow \sum y_k(\mathbf{x}) = 1$$

- ▶ Categorical targets: softmax output activation function

$$y_k(\mathbf{x}, \mathbf{w}) = h^{(L)}(\mathbf{a}^{\text{out}}) = \frac{\exp(a_k^{\text{out}})}{\sum_{j=1}^K \exp(a_j^{\text{out}})}$$

- ▶ Maximum Likelihood/minimum negative log likelihood:

$$E(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln(y_k(\mathbf{x}_n, \mathbf{w}))$$

Generalized
Bernoulli

Losses overview

To minimize

- Regression
 - Assume Gaussian target distribution
 - NN makes prediction for the mean
 - Output activation is identity
- Binary classification
 - Assume Bernoulli target distribution
 - NN makes prediction for probability for class 1
 - Output activation is logistic sigmoid
- Multi-class classification
 - Assume generalized Bernoulli target distribution
 - NN makes prediction for probability for each class
 - Output activation is soft max function

Least squares errors

Cross-entropy loss

(Multi-class) Cross-entropy loss

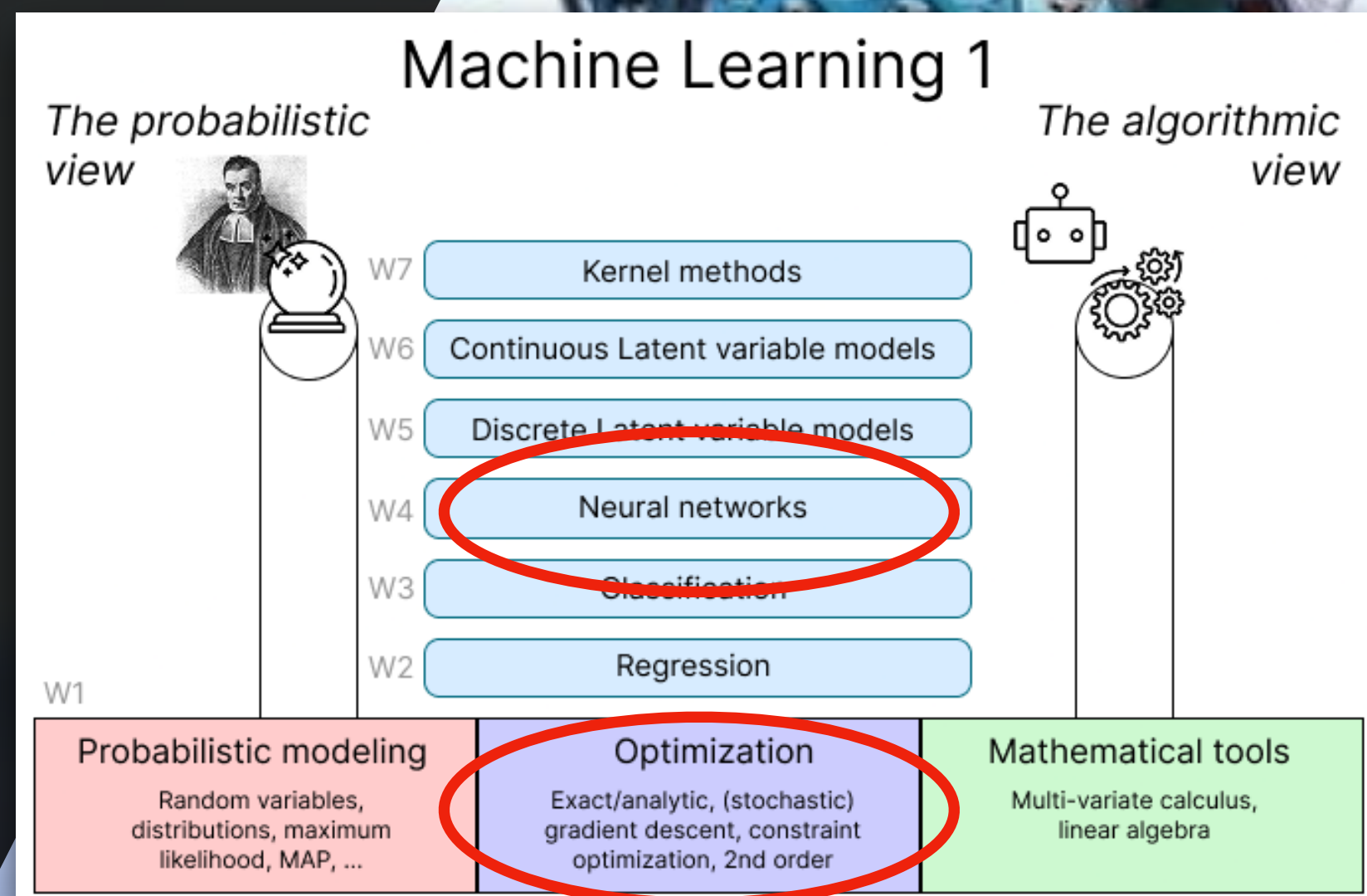


Machine Learning 1

Lecture 8.4 - Supervised Learning
Neural Networks - Training

Erik Bekkers

(Bishop 5.2)



Neural Networks: Parameter Optimization

- ▶ For each task a different loss function $E(\mathbf{w})$
- ▶ Optimal parameters $\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} E(\mathbf{w})$
- ▶ Problem: $E(\mathbf{w})$ is not convex in $\mathbf{w}$, so several local minima can exist.
- ▶ How to reach the global minimum?

*try
via stochastic
gradient descent
(still no guarantee you'll
get there)*

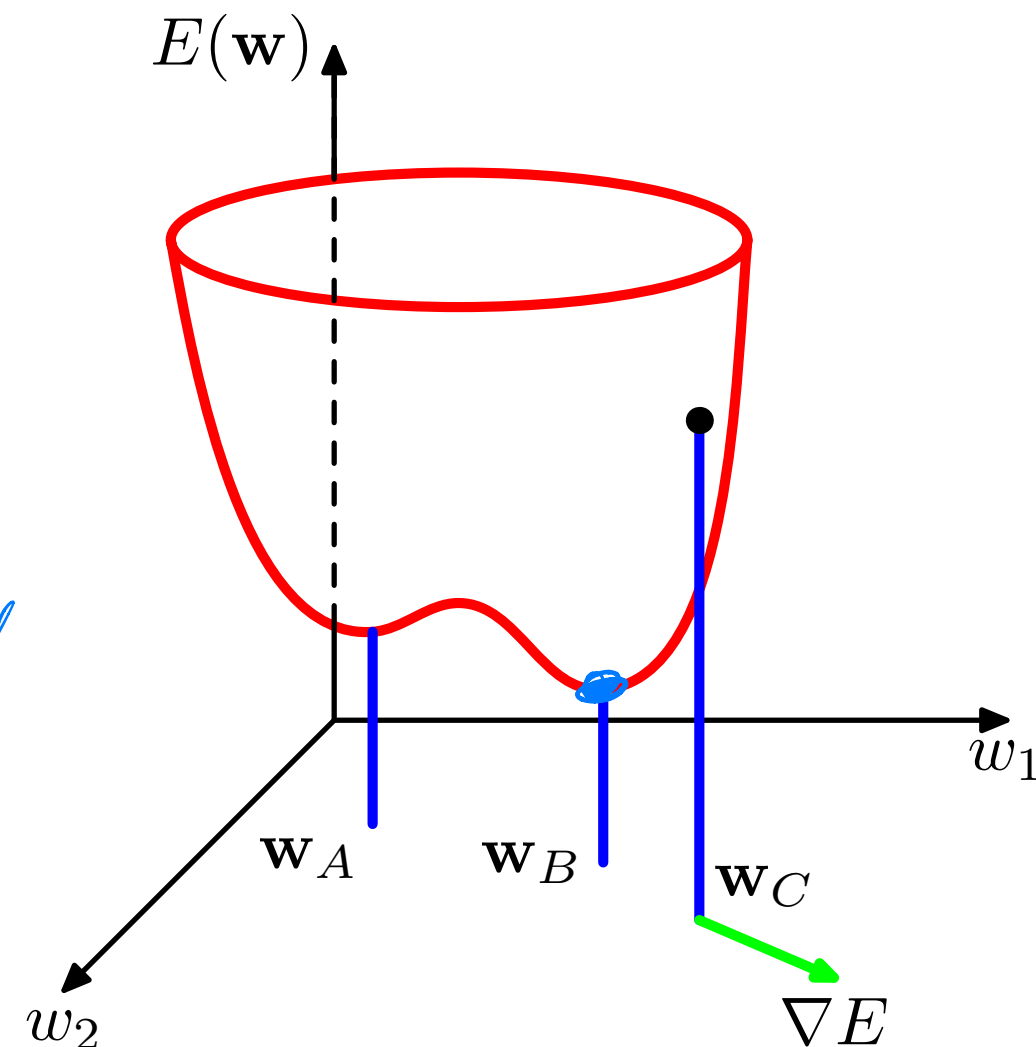


Figure: $E(\mathbf{w})$ as surface in weight space (Bishop 5.4)

Gradient Descent vs. Stochastic Gradient Descent

- ▶ Gradient Descent: $\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E(\mathbf{w}^{(\tau)})$
error on the full dataset
- ▶ Will easily get stuck in local minimum where $\nabla E(\mathbf{w}) = 0$

- ▶ Use $E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w})$ to implement SGD:
 - ▶ Carefully choose learning rate $\eta > 0$
 - ▶ Randomly initialize $\mathbf{w}^{(0)}$
 - ▶ Randomly/sequentially choose $\mathbf{x}_n$ and update $\mathbf{w}$:

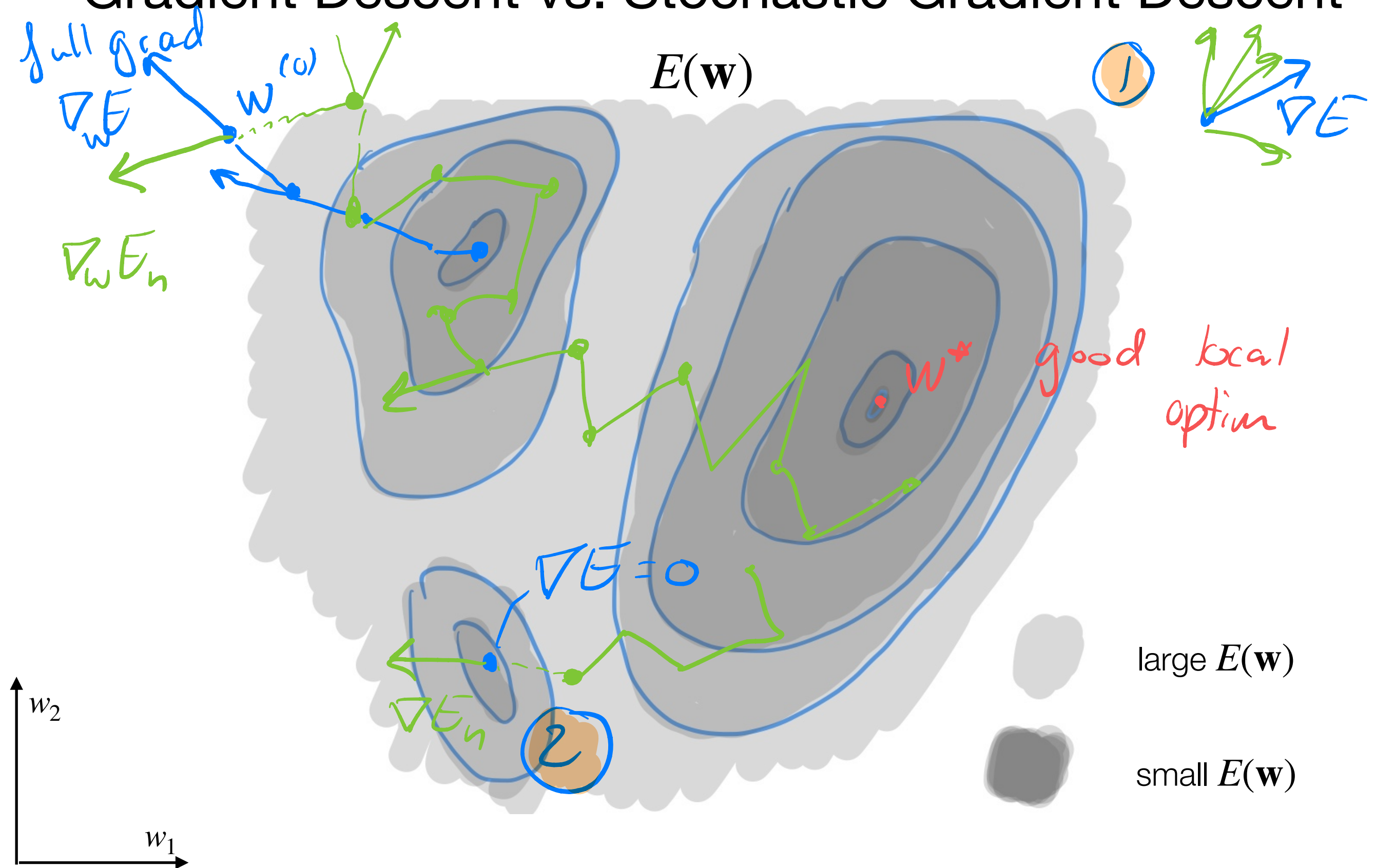
$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E_n(\mathbf{w}^{(\tau)})$$

- ▶ Convergence to area around local minimum
error on a single sample

- ▶ Can also use minibatches $\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla \sum_{i=1}^M E_i(\mathbf{w}^{(\tau)})$

because the gradients are approx. of the full error

Gradient Descent vs. Stochastic Gradient Descent



Gradient Descent vs. Stochastic Gradient Descent

- ▶ About the learning rate:
 - ▶ If learning rate is too small: slow convergence
 - ▶ If learning rate is too large: oscillations around local minimum
 - ▶ Use **learning rate scheduling** with smaller learning rate over time
- ▶ Why SGD over full batch gradient descent

1

- ▶ At the beginning of learning all gradients $\nabla E_n(\mathbf{w})$ will roughly point in the same general direction. Full batch gradient descent computes redundant number of gradients!

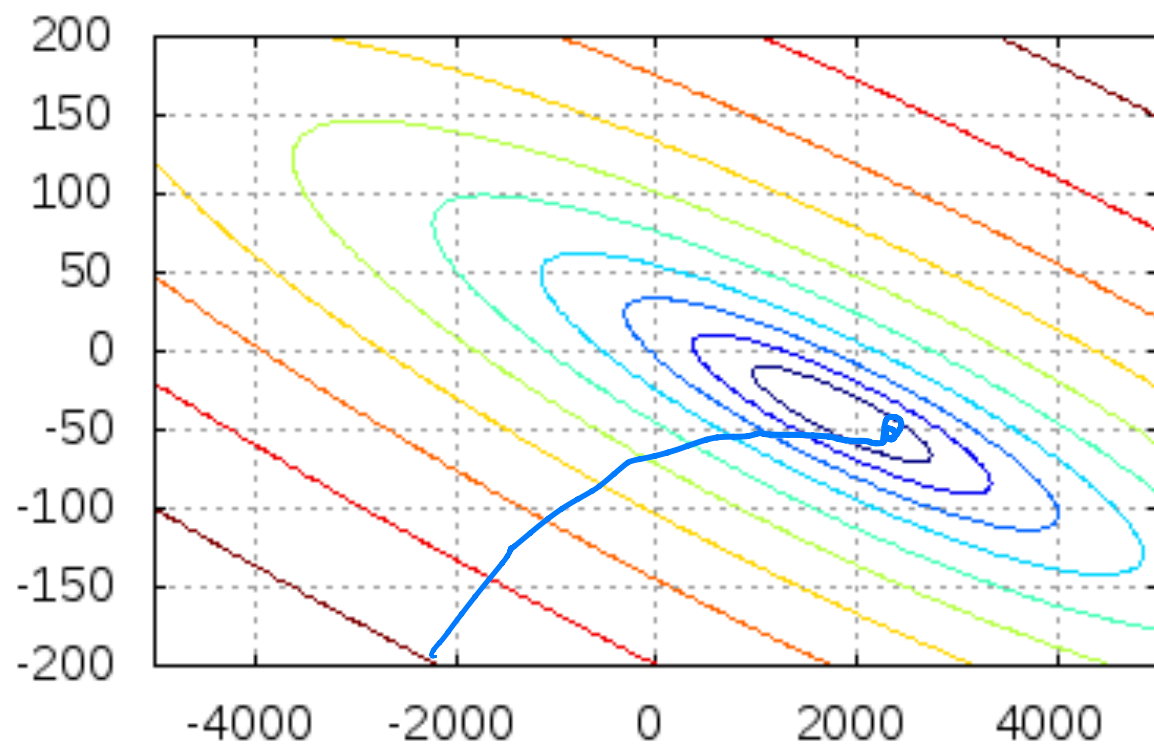
Efficient + accurate enough

2

- ▶ SGD is more likely to escape a local minimum since $\nabla E(\mathbf{w}) = 0$ does **not** necessarily imply $\nabla E_n(\mathbf{w}) = 0$!

Gradient Descent vs. Stochastic Gradient Descent

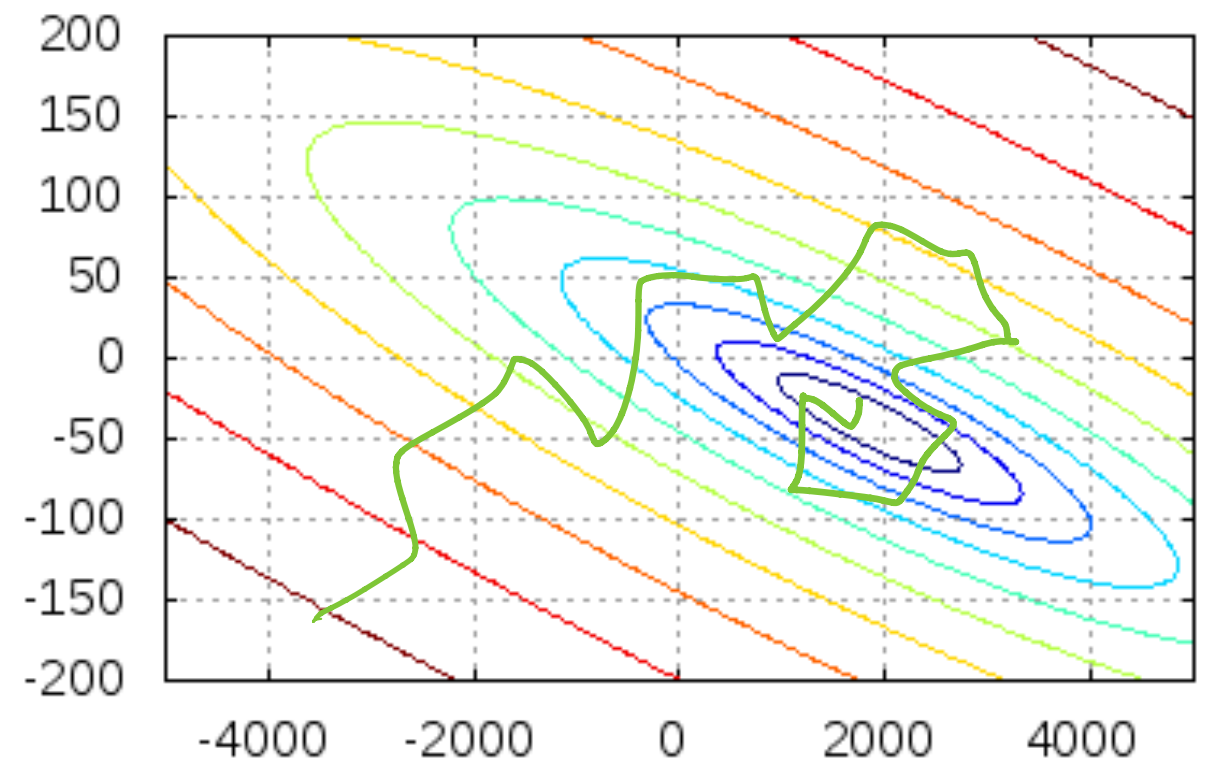
GD



$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E(\mathbf{w}^{(\tau)})$$

- + requires less iterations
- each iteration is slow to compute

SGD



$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E_n(\mathbf{w}^{(\tau)})$$

- requires more iterations
- + each iteration is fast to compute

Example: Test Errors and Local Minima

Restart for different random initial $\mathbf{w}^{(0)}$ to end up in different local minima

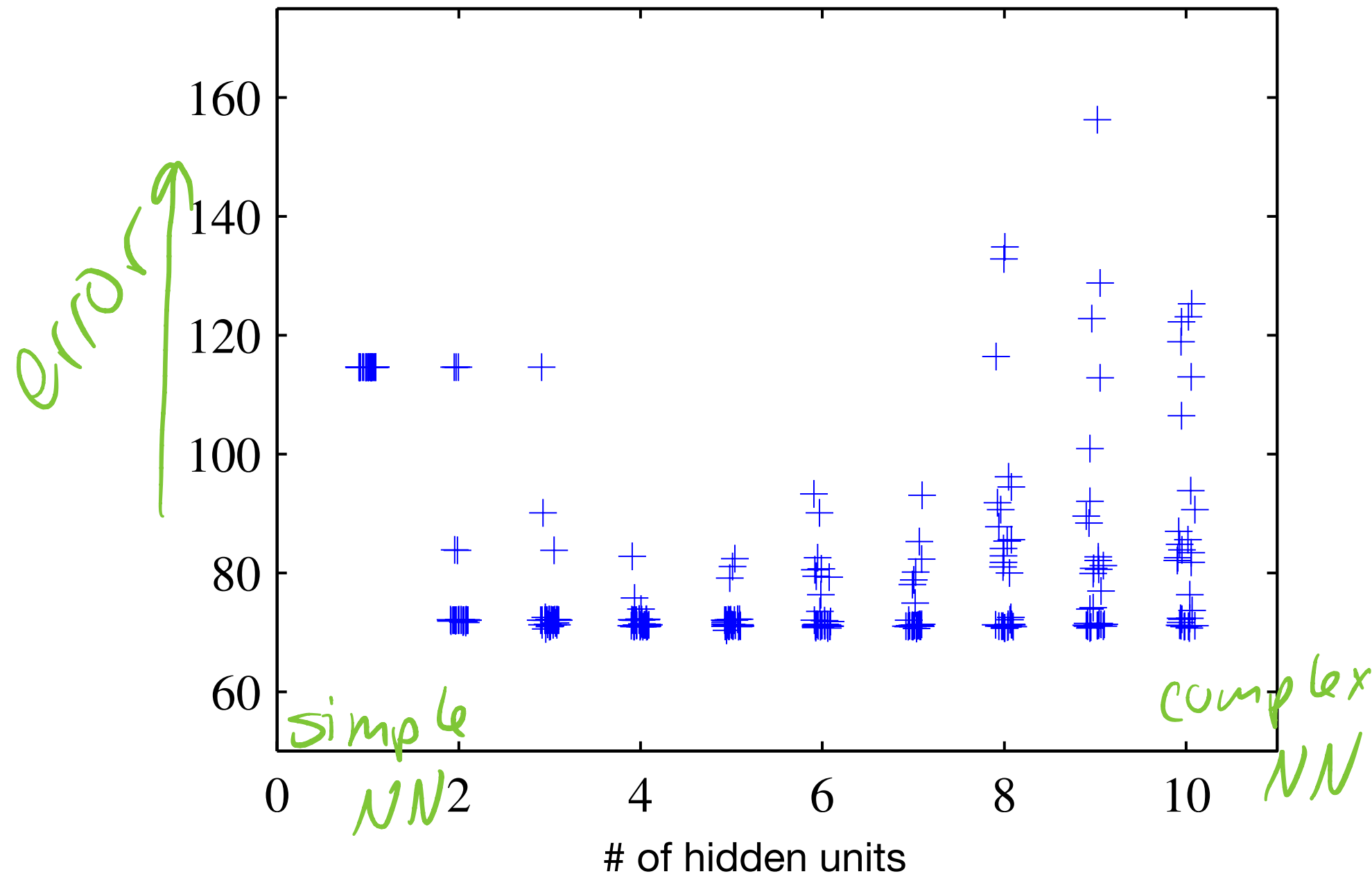


Figure: sum-of-squares test error vs. network size (# of hidden units) for 30 random starts each (Bishop 5.10)

Example: Test Errors and Local Minima

Restart for different random initial $\mathbf{w}^{(0)}$ to end up in different local minima

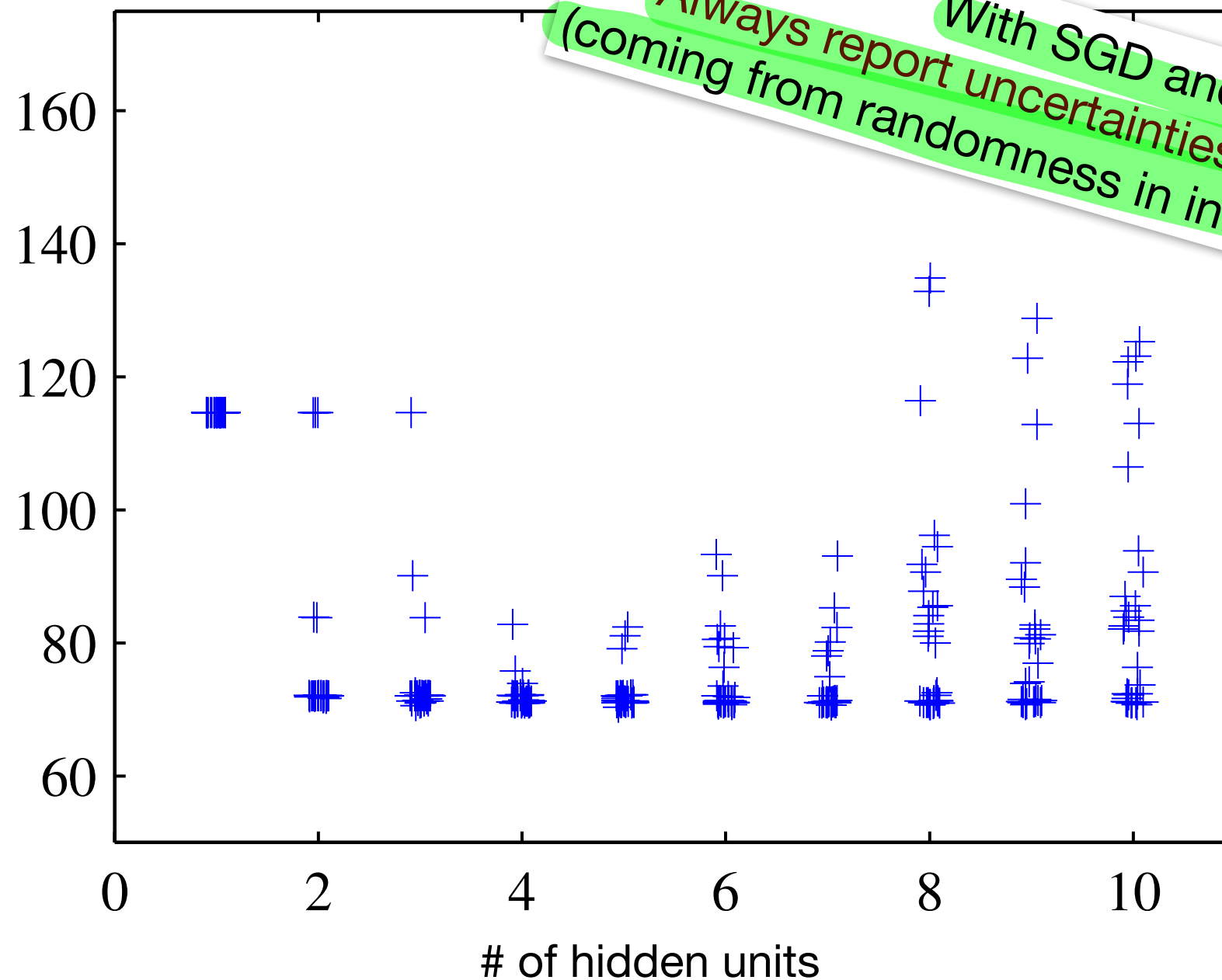


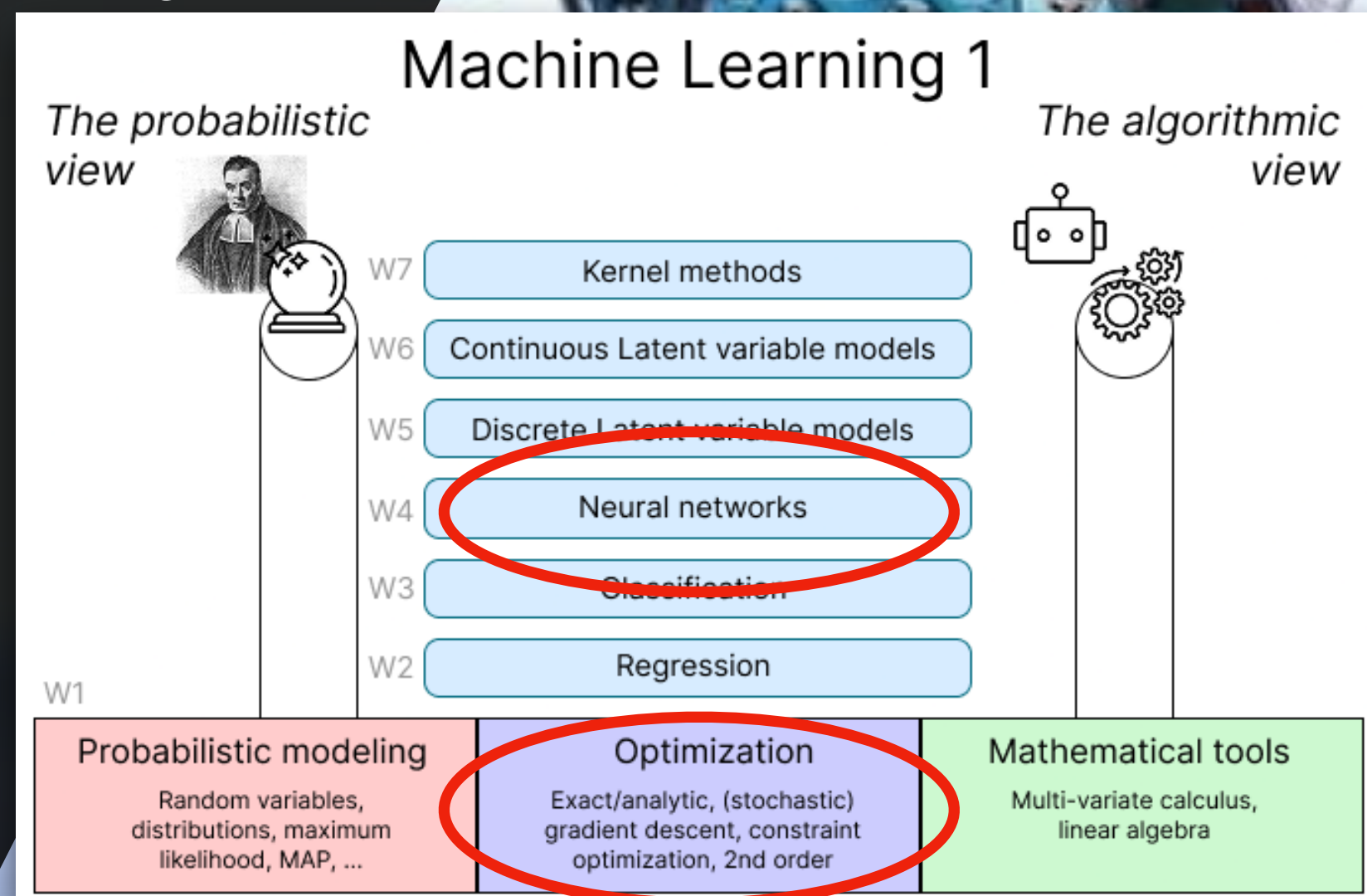
Figure: sum-of-squares test error vs. network size (# of hidden units) for 30 random starts each (Bishop 5.10)

Machine Learning 1

Lecture 8.5 - Supervised Learning
Neural Networks - Error Backpropagation

Erik Bekkers

(Bishop 5.3)



1-dim: $f(g(x)) \Rightarrow \frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial x}$

$$(g_1(x), \dots, g_D(x)) \mapsto f(g_1(x), \dots, g_D(x))$$

$$f(a_1(x), a_2(x), \dots, a_D(x))$$

$$\frac{\partial f(g_1(x), \dots, g_d(x))}{\partial x} = \sum_d \frac{\partial f(g_1(x), \dots, g_D(x))}{\partial g_d(x)} \frac{\partial g_d(x)}{\partial x}$$

$$\frac{\partial f}{\partial g_1} \frac{\partial g_1}{\partial x} + \frac{\partial f}{\partial g_2} \frac{\partial g_2}{\partial x} + \dots$$

Neural Nets: Gradient of Error functions

- Use $E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w})$

- Evaluate $\frac{\partial E_n(\mathbf{w})}{\partial \mathbf{w}}$

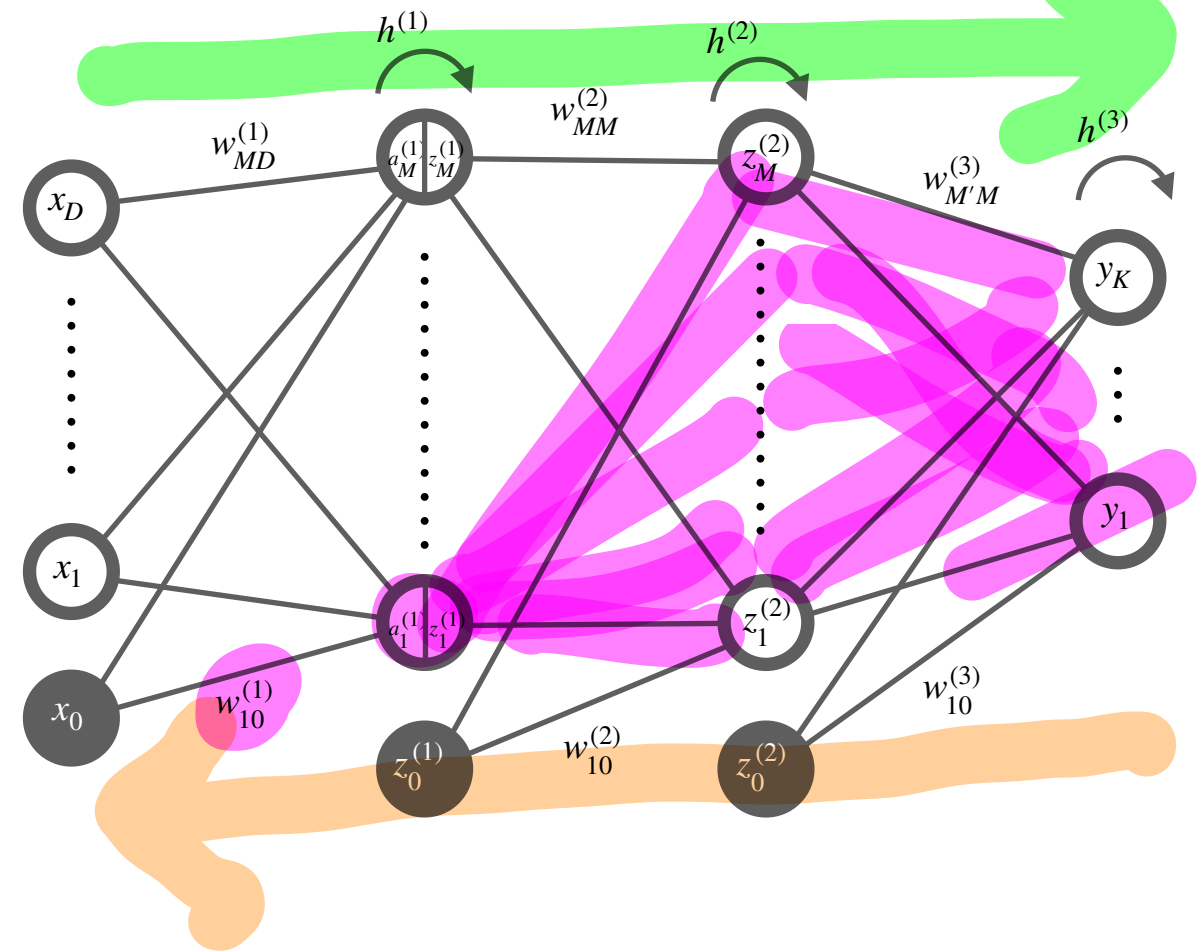
- For general feed-forward network:

- Output/hidden activations: $a_j^{(l)} = \sum_i w_{ji}^{(l)} z_i^{(l-1)}$

- Output/hidden units: $z_j^{(l)} = h^{(l)}(a_j^{(l)})$

- Forward propagation: Compute all a_j and z_j

- Back propagation: Compute all derivatives $\frac{\partial E_n}{\partial w_{ji}^{(l)}}$



Neural Nets: Gradient of Error functions

- ▶ **Back propagation**: Compute all derivatives $\frac{\partial E_n}{\partial w_{ji}^{(l)}}$ corresponding to input $\mathbf{x}_n$
- ▶ E_n only depends on w_{ji} through activation: $a_j^{(l)} = \sum_i w_{ji}^{(l)} z_i^{(l-1)}$

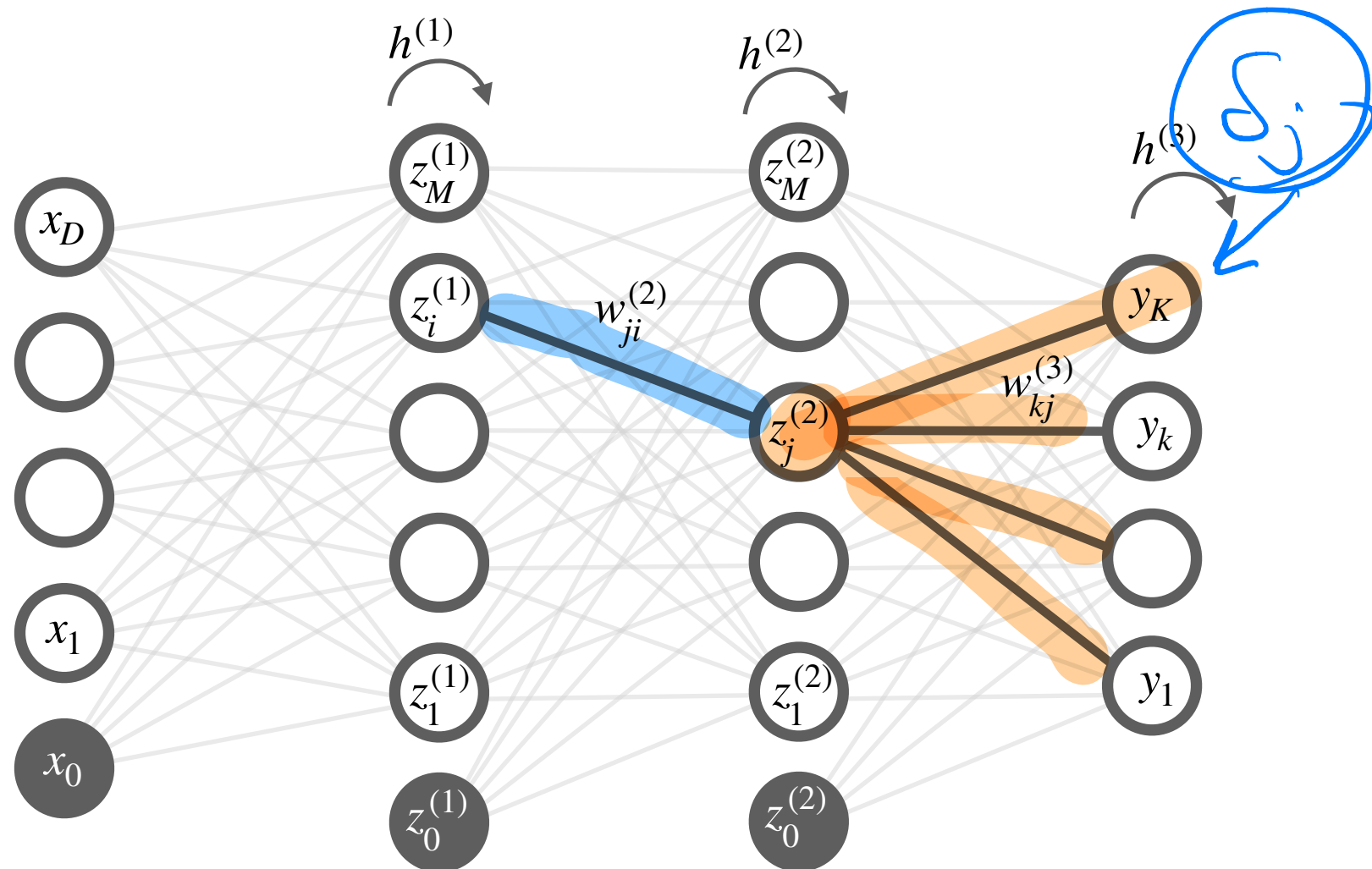
$$\frac{\partial E_n}{\partial w_{ji}^{(l)}} = \frac{\partial E_n}{\partial a_j^{(l)}} \frac{\partial a_j^{(l)}}{\partial w_{ji}^{(l)}}$$

- ▶ Define “node error”

$$\delta_j = \frac{\partial E_n}{\partial a_j}$$

- ▶ Then

$$\frac{\partial E}{\partial w_{ji}^{(l)}} = \delta_j \cdot \frac{\partial a_j^{(l)}}{\partial w_{ji}^{(l)}}$$



Neural Nets: Gradient of Error functions

- ▶ Back propagation: - First compute all δ_j
- Then update all derivatives $\frac{\partial E_n}{\partial w_{ji}}$

- ▶ We now omit layer indices and identify the layers with the indices i, j , and k

$$z_i^{(l-2)} = z_i$$

$$z_j^{(l-1)} = z_j$$

$$z_k^{(l)} = z_k$$

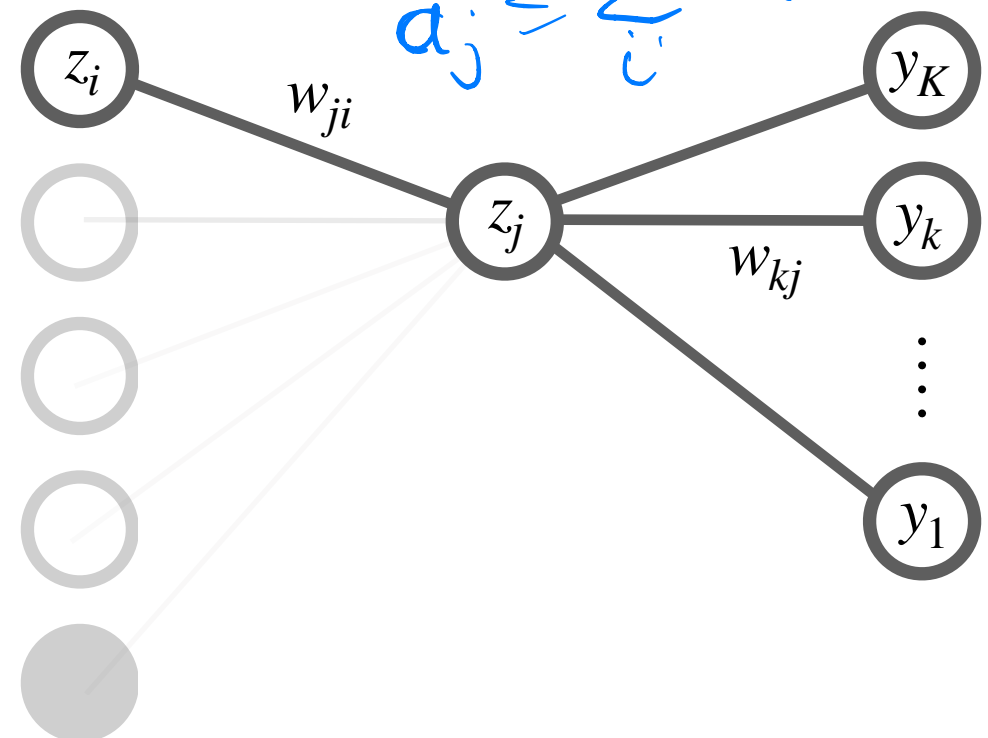
$l-2: i$ $l-1: \text{index } j$ $l: \text{use index } k$

$$a_j = \sum_i w_{ji} z_i$$

- ▶ Now let's compute

$$\frac{\partial a_j}{\partial w_{ji}} = \frac{\partial}{\partial w_{ji}} \left(\sum_i w_{ji} z_i \right)$$

$$= z_i$$



Neural Nets: Gradient of Error functions

- Back propagation: - First compute all δ_j

- Then update all derivatives $\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i$

- We omit layer indices and identify the layers with the indices i, j , and k

- Now let's compute

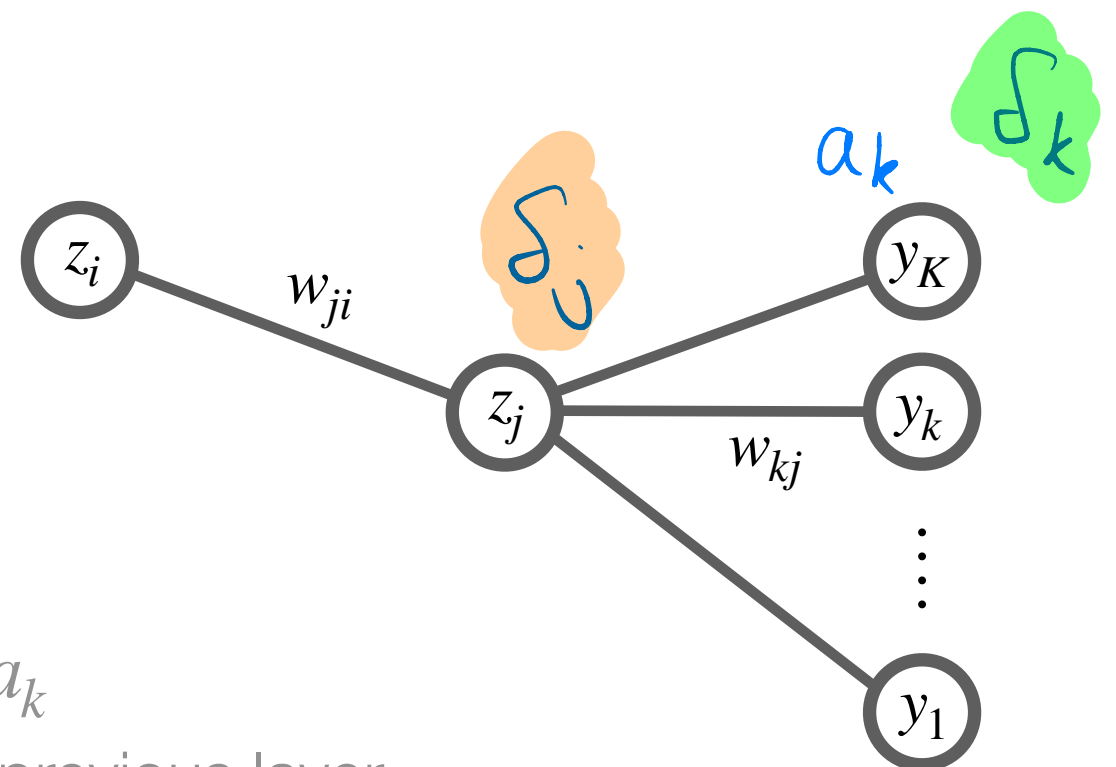
$$\delta_j \equiv \frac{\partial E}{\partial a_j} = \sum_k \frac{\partial E}{\partial a_k} \cdot \frac{\partial a_k}{\partial a_j}$$

$$= \sum_k \delta_k \frac{\partial a_k}{\partial a_j}$$

$E(a_1, \dots, a_K)$ depends on output activations a_k

Outputs $a_k(a_1, \dots, a_J)$ in turn depend on a_j of previous layer

Use multi-dimensional chain rule!



Neural Nets: Gradient of Error functions

- ▶ Back propagation: - First compute all δ_j
 - Then update all derivatives $\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i$
- ▶ We omit layer indices and identify the layers with the indices i, j , and k
- ▶ So $\delta_j = \sum_k \delta_k \frac{\partial a_k}{\partial a_j}$, then let's compute $\frac{\partial a_k}{\partial a_j}$

$$\frac{\partial E}{\partial a_k}$$

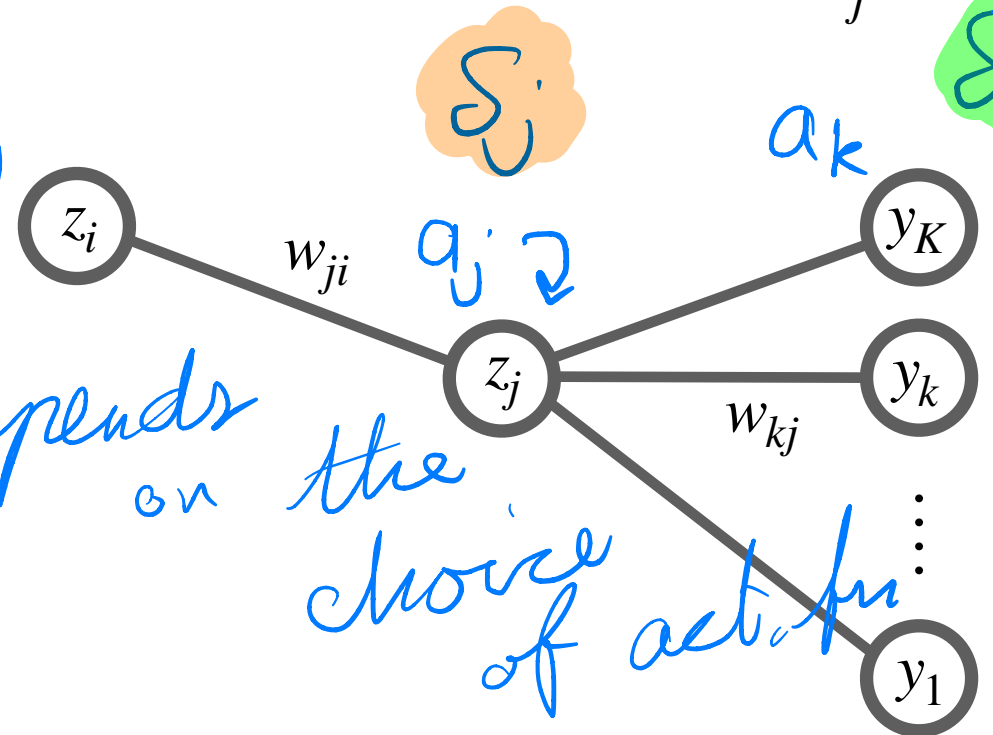
(Recall: $a_k = \sum_j z_j w_{kj}$)

$$\frac{\partial a_k}{\partial a_j} = \frac{\partial}{\partial a_j} \left(\sum_{j'} w_{kj'} h(a_{j'}) \right)$$

$$= w_{kj} h'(a_j)$$

▶ Thus

$$\delta_j = h'(a_j) \cdot \sum_{k'} \delta_{k'} w_{k'j}$$



Forward and Backward Propagation

Forward propagation:

- For input $\mathbf{x}_n$ compute all hidden and output activations a_k and units z_k .

$$a_j = \sum_i w_{ji} h(a_i)$$

Backward propagation:

- Compute δ_k for all output units.

$$\delta_k = \frac{\partial E}{\partial y_k} = (y_k - t_k)$$

- Compute δ_j for all hidden units through back-prop

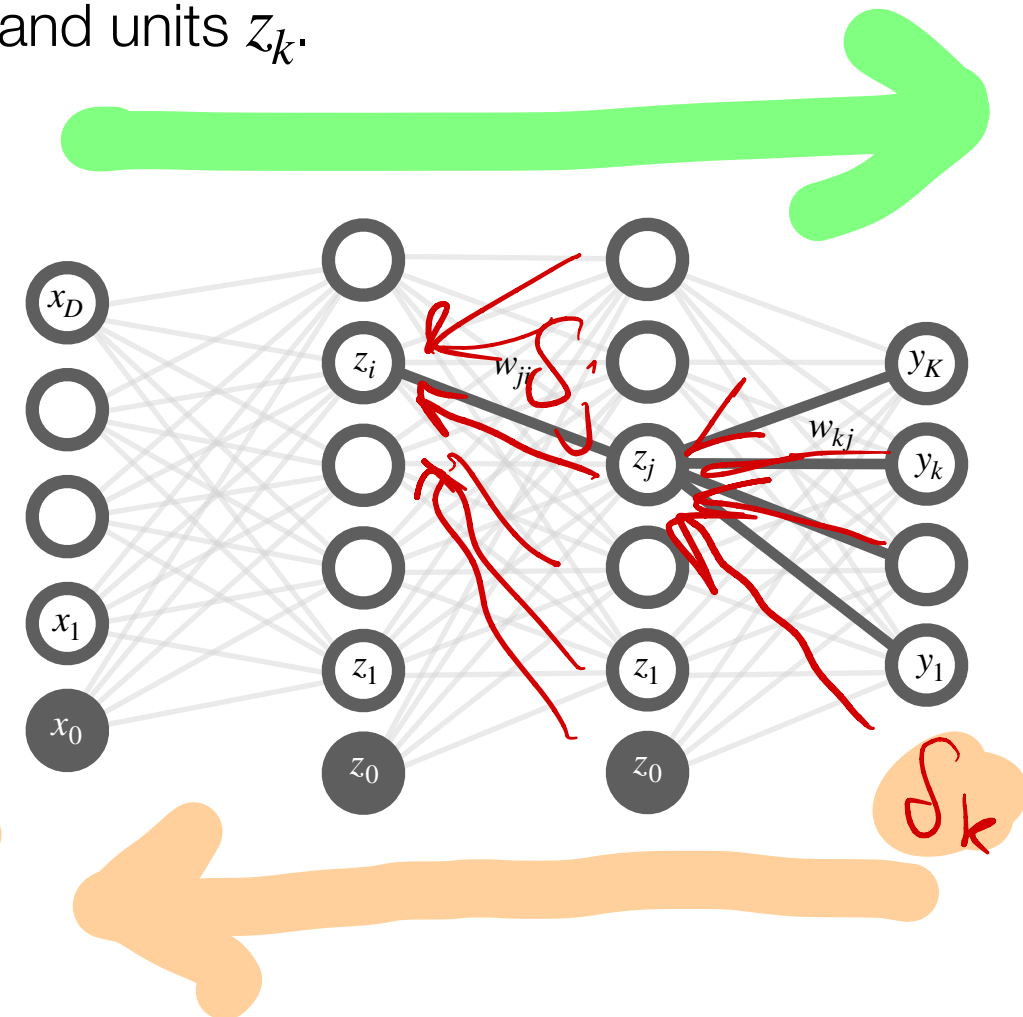
(Careful with skip connections!) $\delta_j = h'(a_j) \cdot \sum_k w_{kj} \delta_k$

- Compute derivatives

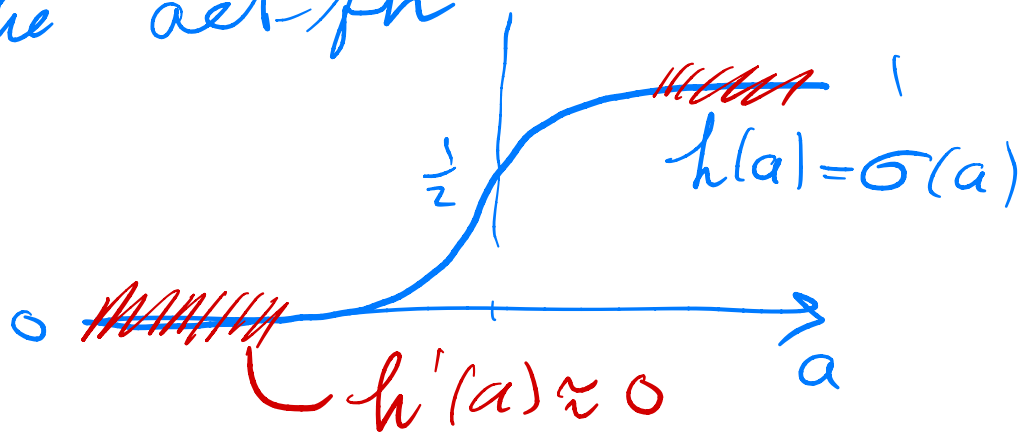
$$\frac{\partial E}{\partial w_{ji}} = \delta_j \cdot z_i$$

Iterative weight updates:

$$SGD: w_{ji}^{(\tau+1)} = w_{ji}^{(\tau)} - \eta \delta_j z_i$$

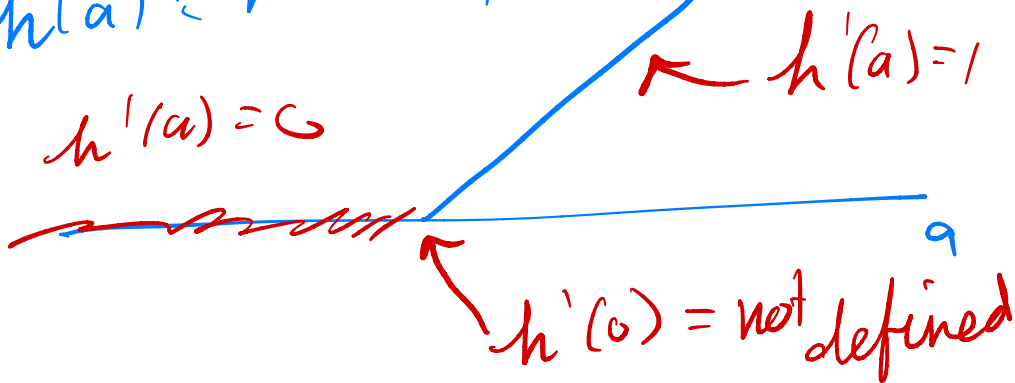


Backprop depends on $h'(a)$, the derivative of the act-fn



Better choice: ReLU

$$h(a) = \max(0, a)$$



Forward and Backward Propagation (with skip connection)

- Consider NN with skip connection (skipping at most 1 layer) with index notation:
 - i for the nodes that are used twice, once from i to j and once from i to k
 - j for the “regular” nodes
 - k for the “output nodes” of such a skip-connect layer

- The forward pass of this layer

$$a_j = \sum_i h(a_i) w_{ji}$$

$$a_k = \sum_j h(a_j) w_{kj} + \sum_i h(a_i) w_{ki}$$

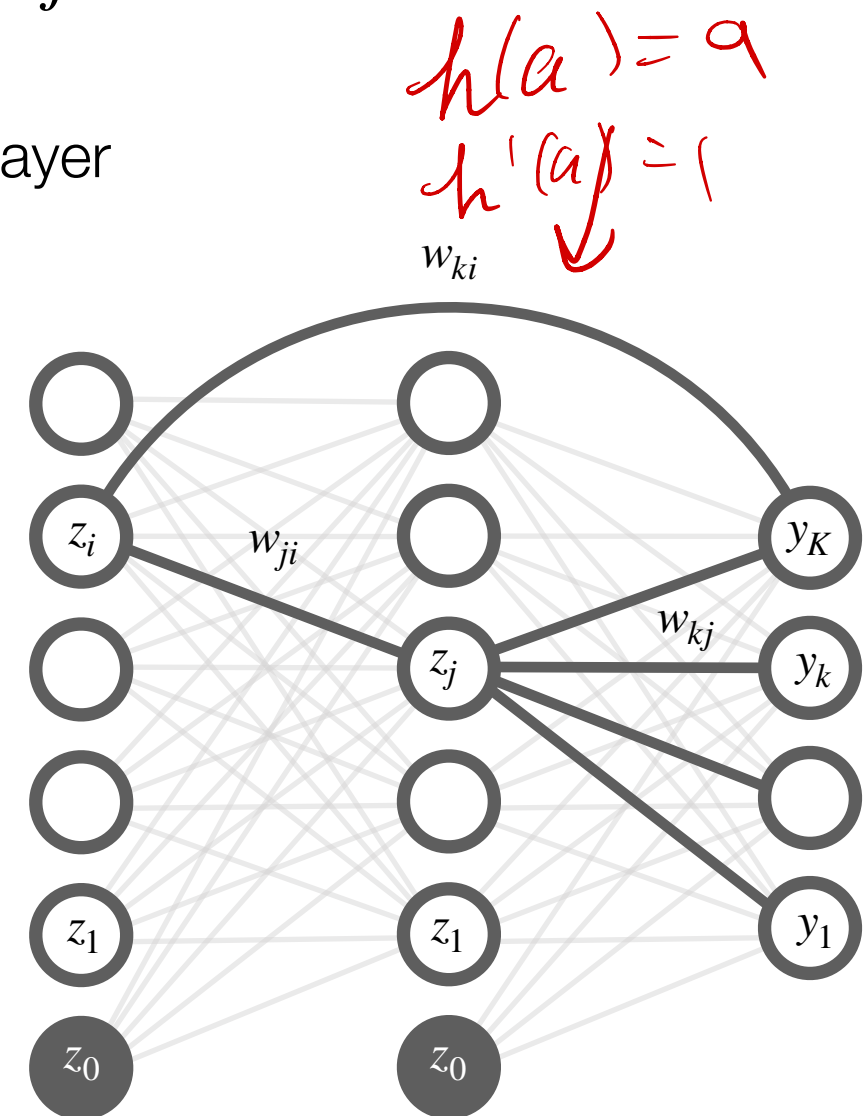
- Then we have the following update rules

$$\delta_j = h'(a_j) \sum_k \delta_k w_{kj} \quad (\text{the usual update rule})$$

$$\delta_i = \delta_i^{(j)} + \delta_i^{(k)}, \quad \text{with} \quad \delta_i^{(j)} := h'(a_i) \sum_j \delta_j w_{ji}$$

and

$$\delta_i^{(k)} := h'(a_i) \sum_k \delta_k w_{ki}$$



Forward and Backward Propagation (with skip connection)

► Derivation

$$\begin{aligned}\delta_i &= \frac{\partial E}{\partial a_i} = \sum_k \frac{\partial E}{\partial a_k} \frac{\partial a_k}{\partial a_i} \\&= \sum_k \delta_k \left(\sum_j \frac{\partial a_k}{\partial a_j} \frac{\partial a_j}{\partial a_i} + \sum_{i'} \frac{\partial a_k}{\partial z_{i'}} \frac{\partial z_{i'}}{\partial a_i} \right) \\&= \sum_k \delta_k \left(\sum_j h'(a_j) w_{kj} h'(a_i) w_{ji} + w_{ki} h'(a_i) \right) \\&= h'(a_i) \sum_j \left(h'(a_j) \sum_k \delta_k w_{kj} \right) w_{ji} + h'(a_i) \sum_k \delta_k w_{ki} \\&= h'(a_i) \sum_j \delta_j w_{ji} + \delta_i^{(k)} \\&= \delta_i^{(j)} + \delta_i^{(k)}\end{aligned}$$

Starting the backpropagation

- For regression: $E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(\mathbf{x}_n, \mathbf{w}) - t_n\}^2$

sh = identity

$$y(\mathbf{x}_n, \mathbf{w}) = y_n = a^{out}$$

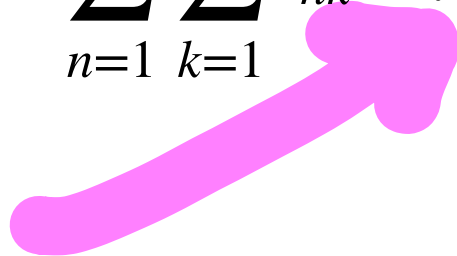
$y = h(a^{out})$

$$\delta^{out} = \frac{\partial E_n}{\partial a^{out}} = y_n - t_n$$

cross-entropy

- For classification with K classes: $E(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln y_k(\mathbf{x}_n, \mathbf{w})$

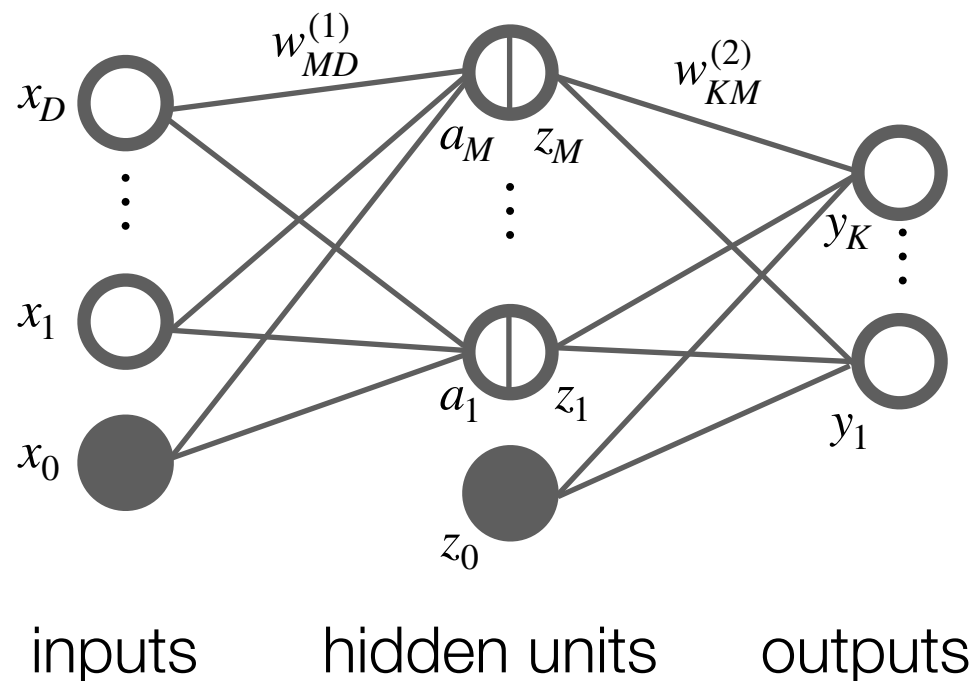
$$y_k(\mathbf{x}_n, \mathbf{w}) = y_{kn} = \frac{\exp(a_k^{out})}{\sum_{j=1}^K \exp(a_j^{out})}$$



$$\delta_k^{out} = \frac{\partial E_n}{\partial a_k^{out}} = y_{kn} - t_{kn}$$

Example: Backpropagation with tanh

- Two layer neural network:



- Regression with K outputs: $y_k = a_k^{(2)}$
- Hidden units: $z_j = h(a_j) = \tanh(a_j^{(1)})$
- Activation function $h(a) = \tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}$
- Has derivative $h'(a) = 1 - h(a)^2$
- Error function $E = \frac{1}{2} \sum_k (y_k - t_k)^2$

- After forward propagation, compute:

$$\delta_k^{out} = y_k - t_k$$

- Backpropagate using:

$$\delta_j = (1 - z_j^2) \sum_{k=1}^K w_{kj}^{(2)} \delta_k^{out}$$

- Update weights in first and second layer using:

$$\frac{\partial E_n}{\partial w_{ji}^{(1)}} = \delta_j x_i \quad \text{and} \quad \frac{\partial E_n}{\partial w_{kj}^{(2)}} = \delta_k^{out} z_j$$